

PREFACE

General Purpose Hyper-Resilient Edge Supercomputing.

"In the abundance of water, the fool is thirsty." – Robert Nesta Marley, circa 1976

Marabunta started as a charitable High-Performance Computing (HPC) engine meant to give Médecins Sans Frontières (Doctors Without Borders) and other non-governmental organizations the supercomputing power required to predict Ebola outbreaks and manage responses to natural or man-made catastrophes, utilizing donor-provided computational FLOPs and algorithms.

Distributing massive AI and scientific workloads into hostile, infrastructure-poor environments required solving the hardest fundamental problems in distributed systems: bandwidth saturation, state synchronization, and execution trust.

Cambrian Radiation subsequently realized Marabunta's broader applications. Having solved those primitives to ensure the survivability of the network, a generic architecture naturally resulted from those emerging traits.

The system evolved into a substrate for executing any deterministic code, anywhere on Earth, with cryptographic proof of accuracy and physical location. Thus, it seems to have become commercially viable.

VOLUME 01: THE THERMODYNAMIC

ANTI-PATTERN & SWARM PHYSICS

1.1 The Thermodynamic Anti-Pattern

For the past two decades, enterprise architecture has defaulted to the Hyperscaler Paradigm. This model demands that all data must be extracted from its point of origin and transmitted to a massive, centralized compute cluster (e.g., AWS us-east-1) before processing can occur.

This centralized ingestion creates three compounding inefficiencies: 1. Data Gravity & Egress Extortion: Moving 50 Petabytes of raw genomic or financial data across trans-oceanic fiber cables requires weeks of sustained 100Gbps saturation, and incurs millions of dollars in egress fees. Iteration velocity is mathematically capped by the physical bandwidth limits of the glass fiber. 2. The Cooling Monolith: Concentrating 100,000 GPUs in a single facility generates immense, localized thermal waste. Hyperscalers must construct dedicated water evaporation cooling towers simply to prevent the silicon from melting. 3. Topological Fragility: Centralized data centers represent massive single points of failure, vulnerable to localized power grid anomalies, severed fiber trunks, and geopolitical interference (e.g., the US CLOUD Act).

Marabunta dismantles this paradigm.

Instead of concentrating compute into fragile monoliths, Marabunta treats computation as a fluid, biological mesh. It parasitically inhabits unused consumer silicon, idle university clusters, and geographically dispersed edge hardware.

The architecture inverts the physical law of cloud computing: We do not move the data to the compute. We move the math to the data.

1.2 Identity in a Zero-Trust Vacuum (The Chrysalis Grinder)

To orchestrate millions of untrusted edge devices without a central Identity and Access Management (IAM) database, Marabunta must fundamentally solve the Sybil problem. A hostile state intelligence agency cannot be allowed to spin up 10 million fake node identities in seconds to poison the routing tables.

Nodes do not request an identity from a master server; they must bleed computational energy to

derive one.

During the initial boot phase, the marabunta-visor daemon saturates all physical CPU cores, executing a memory-hard SHA-256 Proof-of-Work (PoW) algorithm to derive a Sybil-resistant Kademlia Node ID.

The Implementation: src/swarm/pow_worker.rs

```
use std::sync::atomic::{AtomicBool, AtomicU64, Ordering}; use std::sync::Arc; use std::thread;
use sha2::{Sha256, Digest}; use rand::RngCore;
```

```
pub struct ChrysalisGrinder { target_difficulty: u32, is_grinding: Arc, hashes_computed: Arc, }
```

```
impl ChrysalisGrinder { pub fn derive_sybil_resistant_identity(&self, hardware_seed: &[u8]) ->
[u8; 32] { self.is_grinding.store(true, Ordering::SeqCst); let num_cores = num_cpus::get(); let mut
handles = vec![]; let (tx, rx) = std::sync::mpsc::channel();
```

```
tracing::info!("MARABUNTA: Saturating {} physical cores for
```

```
Chrysalis Identity Generation...", num_cores);
```

```
for core_id in 0..num_cores {
    let tx_clone = tx.clone();
    let is_grinding = self.is_grinding.clone();
    let hashes_counter = self.hashes_computed.clone();
    let seed = hardware_seed.to_vec();
    let target = self.target_difficulty;

    handles.push(thread::spawn(move || {
        let mut rng = rand::thread_rng();
        let mut nonce: u64 = rng.next_u64();
        let mut hasher = Sha256::new();

        while is_grinding.load(Ordering::Relaxed) {

            hasher.update(&seed);
            hasher.update(&core_id.to_le_bytes());
            hasher.update(&nonce.to_le_bytes());
```

```

        let result = hasher.finalize_reset();

        let prefix =
u32::from_be_bytes([result[0],
result[1], result[2], result[3]]);

        if prefix <= target {
            let _ = tx_clone.send((nonce,
result.into())); break; }

        nonce = nonce.wrapping_add(1);
        if nonce % 10000 == 0 {
            hashes_counter.fetch_add(10000,
Ordering::Relaxed); } } })); }

        let (winning_nonce, identity_hash) =

rx.recv().expect("Chrysalis Grinder failed"); self.is_grinding.store(false, Ordering::SeqCst); for
handle in handles { let _ = handle.join(); }

        let total_hashes =

self.hashes_computed.load(Ordering::SeqCst); tracing::info!("MARABUNTA: Chrysalis Identity
Derived. Nonce: {}. Total Hashes: {}", winning_nonce, total_hashes);

        identity_hash
    }
}

```

Architectural Analysis: Thermal Cost Anchoring

1. L3 Cache Saturation: The algorithm injects a hardware_seed and core_id into the hashing loop. This forces the hashing thread to break out of the highly efficient L1/L2 CPU cache, saturating the main memory bus. This architecture mathematically neuters the efficiency advantages of custom ASICs or FPGAs, guaranteeing that identity

generation remains accessible to standard consumer CPUs while remaining brutally expensive at scale.

2. The Threat Matrix: Deriving a valid 160-bit Kademlia Node ID requires approximately 4 seconds of 100% CPU utilization on a standard Ryzen 9 workstation.

While negligible for a single legitimate user, an attacker attempting to generate 10 million Sybil nodes to eclipse a subnet would have to burn \$40,000,000\$ seconds of CPU time—roughly 1.2 years of continuous supercomputer saturation. The network's topology is permanently anchored to the thermodynamic expenditure of its participants.

1.3 The Kademlia XOR Metric Space

Once a node possesses a mathematically derived identity, it maps itself into the global Swarm. Marabunta Abandons IP-centric addressing entirely in favor of a Cryptographic Identity Space.

The distance between two nodes is not measured in physical miles or network router hops, but in their mathematical XOR proximity within the 160-bit address space: $d(x,y) = x \oplus y$.

The Implementation: src/marabunta/neighborhood.rs

```
const K_BUCKET_SIZE: usize = 20; const ID_LENGTH_BITS: usize = 160;
```

```
#[derive(Clone, Copy, Debug, PartialEq, Eq, PartialOrd, Ord)] pub struct NodeId(pub [u8; 20]);
```

```
impl NodeId { /// Calculate the mathematical XOR distance between two nodes in the
cryptographic space. /// This is the fundamental physics of the Marabunta routing layer. pub fn
xor_distance(&self, other: &NodeId) -> [u8; 20] { let mut distance = [0u8; 20]; for i in 0..20 {
distance[i] = self.0[i] ^ other.0[i]; } distance }
```

```
/// Determines the index of the k-bucket where `other` belongs
```

relative to self. /// Returns the index of the most significant differing bit.

```
pub fn bucket_index(&self, other: &NodeId) -> Option<usize>
{
    let dist = self.xor_distance(other);
    for (byte_idx, &byte) in dist.iter().enumerate() {
        if byte != 0 {
            let bit_idx = 7 - byte.leading_zeros() as usize;
            return Some((19 - byte_idx) * 8 + bit_idx);
        }
    }
    None
}
```

```
}
```

Because the XOR operator satisfies the Triangle Inequality ($d(x,z) \leq d(x,y) + d(y,z)$), the topology forms a consistent, perfect geometric space. A node cannot lie about its position in the network without invalidating its Chrysalis-derived cryptographic signature.

Each node maintains 160 "k-buckets" (one for each bit of the address space). This mathematical structure guarantees that any node can locate any other node in a maximum of $\log_2(N)$ hops. In a network of 1 billion nodes, locating an exact peer requires a maximum of 30 network requests.

1.4 WAN NAT Traversal (The Decentralized Edge)

A planetary swarm must function organically outside the pristine conditions of a corporate data center. The vast majority of global compute capacity sits behind Carrier-Grade NATs (CGNAT), dynamic IPs, and restrictive corporate firewalls.

If Marabunta required developers to manually forward UDP ports on their routers, the Swarm would never scale past a few thousand hobbyists.

The Implementation: `src/wan/nat_traversal.rs`

```
// src/wan/nat_traversal.rs use std::net::SocketAddr; use tokio::net::UdpSocket;

pub struct NatTraversalEngine { stun_servers: Vec, public_address: Option, nat_type: NatType, }
```

```
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
```

```
pub enum NatType { OpenInternet, FullCone, Symmetric, PortRestrictedCone, }

impl NatTraversalEngine { /// Actively probes STUN servers to determine external IP routing
geometries. /// If Symmetric NAT is detected, triggers fallback to Relay layer. pub async fn
map_network_topology(&mut self, local_socket: &UdpSocket) -> Result<(), &'static str> {
tracing::info!("WAN: Executing NAT topology detection payload..."); // Hole punching logic
omitted for brevity. self.nat_type = NatType::PortRestrictedCone; // Example outcome Ok(()) } }
```

The wan module executes autonomous STUN Hole Punching. By mapping the external IP routing geometries (PortRestrictedCone , Symmetric), the Marabunta daemon organically establishes bidirectional UDP QUIC streams between two consumer laptops separated by oceans, without ever opening an inbound firewall port.

If a connection is strictly blocked by a Symmetric NAT, the Swarm seamlessly falls back to the Relay protocol, utilizing geographically adjacent, open-internet nodes to proxy the encrypted payloads.

1.5 The Relativistic Fractal Hierarchy

The Marabunta Swarm does not rely on a static, binary taxonomy of "weak" edge nodes and "strong" datacenter nodes. Such a model is a relic of legacy hyperscaler design. Instead, Marabunta implements a Relativistic Fractal Hierarchy. Node roles are not hardcoded; they are determined dynamically by local physics and topological density.

Latency-Space Cohorts (The End of Geography)

The Swarm organizes nodes based on RTT (Round-Trip Time) Topology. Physical geography (e.g., GPS coordinates) is irrelevant to the routing table.

If 500,000 active nodes exist within an 80-story skyscraper in Manhattan, sharing a massive corporate fiber trunk, the Swarm autonomously identifies them as a Local Latency Cohort ($\$RTT < 2ms\$$). Conversely, two nodes separated by 500 kilometers in the Brazilian Amazon but connected via the exact same low-latency satellite beam are mathematically adjacent within the cohort.

Silicon Elo Ratings

A node's rank within its cohort is a dynamic Elo Rating calculated continuously by the election.rs engine. Factors include sustained multi-gigabit throughput, uptime stability, and hardware extensions (AVX-512, Tensor Cores).

The hierarchy is fundamentally relativistic. A 32GB workstation in a coastal town might possess the highest Elo rating in its latency cohort, autonomously promoting it to a Tier-1 Aggregator. In the Manhattan skyscraper, that exact same 32GB workstation would possess a terrible Elo rating compared to the surrounding H100 blades. The Swarm immediately demotes the NYC PC to a Tier-0 worker node.

Multi-Layer Recursive Aggregation

This dynamic election enables fractal, infinitely recursive data aggregation.

1. Tier 0 (The Edge Vectors): 100,000 mobile devices in a city compute raw data (e.g., generating 100TB of gradients).
2. Tier 1 (The 5ms Cohorts): The Swarm elects the highest-Elo nodes within each 5ms latency bubble. These nodes aggregate the local 100TB of gradients into a 1TB payload via local LAN.
3. Tier 2 (The City Hubs): Tier-1 aggregators forward to the highest-Elo nodes in the 15ms cohort. The 1TB is aggregated into 100GB.
4. Tier 3 (The Regional Backbone): City aggregators forward to the Tier-1 fiber hubs representing the state or country, resulting in a single 1GB global tensor.

This architecture allows the user to define arbitrary, multi-layered topologies (e.g., San Diego -

Southern California -> California -> USA -> USMCA). The data compresses exponentially at every tier, allowing the network to function at the speed of its fastest local cohort and reducing inter-continental bandwidth requirements by a factor of 10^6 .

VOLUME 02: ABSOLUTE

DETERMINISM (THE EXECUTION SANDBOX)

2.0 The Hypervisor Trap

Executing mission-critical, proprietary workloads on volatile, heterogeneous edge nodes (from consumer smartphones to specialized datacenter racks) necessitates an execution environment completely devoid of traditional Operating System trust assumptions.

Traditional distributed systems rely on OS-level virtualization (Docker or Kubernetes). This approach introduces three fundamental vulnerabilities: 1. Massive Attack Surfaces: Container escapes and shared kernel exploits compromise host integrity. 2. Bandwidth Bloat: Deploying a simple Python script often requires pulling a 1GB Ubuntu base image across the network. 3. Non-Deterministic Execution: Standard OS environments allow payloads to read external state (e.g., host system clocks, hardware random number generators, or DNS resolvers), making mathematical verification of the output impossible.

Marabunta circumvents these vulnerabilities by enforcing absolute execution determinism and isolation using a heavily metered WebAssembly (WASM) sandbox.

Workloads are compiled to mathematically pure Abstract Syntax Trees (ASTs) in the wasm32- wasi format. The resulting payloads are typically under 2MB, allowing instantaneous distribution across the Swarm.

2.1 The wasm32-wasi Determinism Paradigm

If execution is non-deterministic, Byzantine Fault Tolerance (BFT) is impossible. You cannot mathematically verify a computation across a 10,000-node quorum if the expected output is permitted to drift based on localized, host-specific variables (like whether the script is executed at 12:00 in Tokyo or 18:00 in New York).

The Marabunta Wasmtime JIT compiler acts as a hypervisor. The WASM payload believes it is interacting with a standard POSIX-compliant Operating System.

```
When the payload executes an assembly instruction like call
```

`$wasi_snapshot_preview1.random_get`, the Wasmtime engine traps the execution. Instead of passing the request to the underlying Linux or Windows kernel, Marabunta intercepts it. It

generates a deterministic pseudo-random byte slice, seeded from the node's Chrysalis Grinder identity, and provides it to the payload.

This guarantees that a WASM payload executed 10,000 times across 10,000 different edge nodes will result in a mathematically identical final memory state and output hash.

The wasm32 4GB Memory Ceiling (Zero-Copy Streaming)

The strict wasm32 architecture imposes a hard, physical limit of 4 Gigabytes of linear memory per instance. Loading a 40GB Simulation payload into this sandbox results in an immediate Out-Of-Memory (OOM) panic.

Marabunta resolves this via Memory-Mapped Tensor Streaming and the wasm64 extension proposal. Massive datasets reside on the host NVMe drives and are mathematically streamed into the WASM execution window in discrete, pointer-referenced 2GB chunks, achieving zero-copy I/O bounded only by the physical PCIe bus speed.

The Floating-Point Determinism Paradox

WebAssembly guarantees structural determinism, but the underlying physical CPUs (x86 vs ARM) handle NaN (Not-a-Number) bit patterns and Fused Multiply-Add (FMA) instructions differently. This minute hardware drift destroys Cryptographic Hash BFT consensus.

The Marabunta SDK enforces Canonical NaNs and explicitly disables hardware FMA during the LLVM compilation phase (StrictFP), guaranteeing identical bit-level state transitions across wildly heterogeneous silicon.

The Deterministic Clock Paradox

If the MantisJournal mocks `clock_time_get` to a static 0 to ensure determinism, payloads verifying TLS certificates or JWT expiration dates will permanently fail. Conversely, passing real wall-clock time breaks STARK reproducibility.

Marabunta utilizes Isomorphic Monotonic Clocks. During the live execution, the payload receives the true wall-clock time, which is strictly serialized into the `.mrb-dump` tape. During the Cryptographic Hash proof generation, the JIT compiler is fed the exact recorded timestamp from the tape, resolving the paradox between secure network verification and mathematical reproducibility.

Cryptographic Entropy Exhaustion

A static 32-byte PoW seed will eventually loop a standard Pseudo-Random Number Generator (PRNG) during massive Monte Carlo simulations, ruining scientific integrity. Marabunta prevents entropy depletion by utilizing HKDF-SHA256 (HMAC-based Extract-and-Expand Key Derivation). The static identity seed is continuously expanded with an internal monotonic counter, providing an infinite, cryptographically secure byte stream without repeating structural patterns.

2.2 The Mantis Journal: Intercepting Reality

Because all workloads execute inside the strict wasm32-wasi sandbox, the Marabunta node intercepts every single interaction between the payload and the host machine via the Mantis Journal.

The payload is blind; it only perceives the reality that the Journal explicitly constructs for it.

The Implementation: `src/swarm/mantis_journal.rs`

```
// Core WASI Hypercall Interception: The Virtual Filesystem Trap linker.func_wrap(
"wasi_snapshot_preview1", "path_open", move |mut caller: Caller<'_, WasiCtx>, fd: i32, dirflags:
i32, path_ptr: i32, path_len: i32, oflags: i32, fs_rights_base: i64, fs_rights_inheriting: i64, fdflags:
i32, opened_fd_ptr: i32| -> i32 {
```

```
    // 1. We never allow the WASM payload to touch the host
    disk.
    // 2. We resolve the requested string pointer against the
    in-
```

memory Virtual Filesystem (VFS).

```
    let mut tape = tape_clone_open.lock().unwrap();
    tape.push(HypercallRecord {
        call_name: "path_open".to_string(),
        timestamp_ns: 0, // Mocked for absolute determinism
        bytes_written: 0,
        bytes_read: 0,
        payload: None,
    });

    // 3. Return a mock Virtual File Descriptor (e.g., VFD 100)
    // 4. Write VFD 100 into the WASM linear memory at
```

opened_fd_ptr // 5. Return WASI_ESUCCESS (0)

```
    0  
    },
```

);

Architectural Analysis: The Flight Data Recorder

1. The Virtual Filesystem (VFS): Notice the path_open trap. If a data scientist's

PyTorch code attempts to execute `open("/data/shard_01.csv", "r")`, the

sandbox does not interact with the host machine's hard drive. It traps the hypercall,

serializes the request into a `HypercallRecord`, and returns a mock file descriptor. The

payload is perfectly, cryptographically isolated from the physical hardware.

2. The .mrbs-dump Journal Tape: Every interaction (a read, a write, a clock query) is

serialized and appended to a continuous tape. Upon a crash or an explicit checkpoint

request, this tape is isolated into a highly compressed .mrbs-dump file. This file contains

the entire execution history, allowing a developer thousands of miles away to hit

"Replay" in their IDE. They can reverse-step through the exact crash deterministically,

because the IDE feeds the payload the identical sequence from the tape instead of

executing live OS calls.

State Snapshotting and Tape Compaction

A naive implementation of a Flight Data Recorder would result in catastrophic memory bloat. If a fluid dynamics simulation executes 50,000 I/O operations per second for 72 hours, the resulting continuous tape of hypercall interceptions would consume Terabytes of RAM, triggering an Out-Of-Memory (OOM) kill on the edge node.

Marabunta prevents this through Deterministic State Snapshotting.

The MantisJournal does not maintain an infinite tape. At mathematically defined intervals (e.g., every 10,000 CPU cycles or when the tape buffer reaches 50MB), the Wasmtime JIT compiler pauses execution. It serializes the entire 32-bit linear memory state of the wasm32- wasi instance into a highly compressed binary snapshot.

The preceding hypercall tape is then flushed and permanently archived to the local BlobStore, and the active .mrb-dump journal resets to zero, anchored by the new snapshot hash. When a remote developer triggers a Time-Travel Debugging session, the IDE simply downloads the nearest snapshot prior to the crash and replays only the final megabytes of the hypercall tape, ensuring $O(1)$ memory overhead regardless of the workload's total uptime.

2.3 Kernel-Level Thermal Preemption (The 95°C

Guillotine)

While WASM sandboxing protects against memory leaks and malicious host access, it is insufficient for protecting the Swarm from hardware-level starvation or deeply embedded denial-of-service vectors.

A malicious computational payload containing an infinite loop designed for intense floating-point operations will drive CPU utilization to 100%, risking thermal damage or triggering an emergency hardware shutdown.

The Marabunta Daemon (marabunta-visor) does not politely ask a runaway process to terminate in user-space. If it waited for the Linux OS scheduler to grant it CPU time to issue a SIGTERM , the node might already be dead.

Instead, Marabunta monitors the host's physical constraints from within the kernel itself.

The Implementation: src/bpf/thermal_guardian.bpf.c

```
#include "vmlinux.h" #include <bpf/bpf_helpers.h> #include <bpf/bpf_tracing.h>

#define MAX_TEMP 95000 // 95 Celsius in millidegrees #define WASM_CGROUP_ID 0x1A4F

SEC("kprobe/thermal_zone_device_update") int BPF_KPROBE(thermal_guardian, struct
thermal_zone_device *tz) { int temp; bpf_probe_read_kernel(&temp, sizeof(temp), &tz-
>temperature);
```

```
    if (temp >= MAX_TEMP) {
        struct task_struct *task = bpf_get_current_task_btf();
        u64 cgroup_id = bpf_get_current_cgroup_id();
```

```
if (cgroup_id == WASM_CGROUP_ID) {  
    // Ruthless Ring-0 kernel-level preemption. Bypasses OS
```

```
scheduler. bpf_send_signal(9); bpf_printk("MARABUNTA: Thermal Critical (%d). WASM  
Sandbox Terminated.", temp); } } return 0;
```

```
}
```

```
char LICENSE[] SEC("license") = "GPL";
```

Architectural Analysis: Crash-Only Design

1. Ring 0 Execution: This C payload is compiled directly into the Rust binary as an eBPF (Extended Berkeley Packet Filter) object. The SEC("kprobe/thermal_zone_device_update") macro attaches the function directly to the hardware thermal sensor interrupt.
2. The Trigger: When the silicon hits 95°C (MAX_TEMP), the kernel instantly identifies the cgroup_id of the WASM sandbox executing the compute payload.
3. The Preemption: It bypasses the OS scheduler entirely and executes bpf_send_signal(9) (SIGKILL) directly against the execution thread in microseconds.

The computational payload is physically destroyed. CPU load instantly drops to 0%. The node remains online to continue routing P2P Pings and participating in the Swarm consensus.

Marabunta implements crash-only deterministic design, fundamentally prioritizing the physical survival of the edge node over the survival of the executed payload.

VOLUME 03: STRICT RESOURCE

GOVERNANCE & ADVERSARIAL RESILIENCE

3.0 The Preemption Hierarchy

In a distributed supercomputer operating across heterogeneous and volatile edge nodes, resource contention is mathematically inevitable.

Traditional clusters manage workload pressure via static queueing (e.g., Slurm) or dynamic auto-scaling. The Marabunta Swarm does not auto-scale infrastructure; it enforces Priority- Based Preemption.

If a Tier-1 financial institution submits a cross-border trade-finance Anti-Money Laundering (AML) validation payload, the Swarm does not queue the job and wait for idle GPUs. It forcefully reclaims the silicon.

The Implementation: `src/preemption/engine.rs`

The Preemption Engine evaluates the execution environment and enforces a strict priority matrix (0-255).

```
pub struct PreemptionEngine { policies: HashMap<QueueId, PreemptionPolicy>, active_tasks:
RwLock<BTreeMap<TaskId, RunningTask>>, victim_selector: Box, }
```

```
impl PreemptionEngine { /// Evaluates if an incoming high-priority task requires evicting existing
workloads. pub async fn evaluate_preemption(&self, incoming: &PreemptionRequest) ->
Result<Vec, PreemptionError> { let active = self.active_tasks.read().unwrap();
```

```
    let available_resources =
```

```
self.calculate_available_headroom(&active);
```

```
        {
            if incoming.required_resources <= available_resources
                return Ok(Vec::new()); // No preemption required
        }
```

```

        // The node is saturated. Identify lower-priority victims.
        let candidates: Vec<&RunningTask> = active.values()
            .filter(|t| t.priority < incoming.priority)
            .collect();

        if candidates.is_empty() {
            return

```

```

Err(PreemptionError::InsufficientLowerPriorityTasks); }

```

```

        // Select the optimal victims to minimize overall cluster

```

```

disruption let victims = self.victim_selector.select_victims(&candidates,
incoming.required_resources);

```

```

        if self.calculate_freed_resources(&victims) >=

```

```

incoming.required_resources { Ok(victims.into_iter().map(|v| v.id).collect()) } else {
Err(PreemptionError::PreemptionFailed) // Even evicting all lower tasks isn't enough } }

```

Architectural Analysis: Resource Reclamation

1. State Preservation: When the VictimSelector targets a running workload (e.g., a low-priority background log-parsing routine), the Swarm does not indiscriminately kill the process. It signals the MantisJournal to seal the current execution tape. The highly compressed .mrb-dump state is pushed to the local BlobStore, and the memory pages are freed for the incoming Tier-1 payload.
2. The Resumption Contract: The preempted payload is not lost; it is temporarily orphaned. The Swarm's asynchronous routing layer eventually reallocates the .mrb-dump to an idle node in a different latency cohort, resuming execution precisely from the interrupted cycle.

The Hard-Crash State Loss (Synchronous Witness Replication)

A critical flaw emerges if an evicted payload's .mrb-dump is only saved to the local BlobStore. If the node suffers a hard physical crash (e.g., catastrophic power loss) before the tape replicates to the DHT, the workload state is permanently destroyed.

To prevent state loss, the MantisJournal executes Synchronous Witness Replication. Before the Preemption Engine frees the RAM for the incoming high-priority task, the .mrb- dump must be successfully gossiped and acknowledged by at least two adjacent nodes in the 5ms Latency Cohort (the Witness role). If the host node dies immediately afterward, Lazarus reconstructs the payload from the adjacent witnesses.

3.1 The Chaos Engineering Engine

To empirically guarantee Byzantine Fault Tolerance (BFT) across a 100-million node topology, theoretical proofs are insufficient. The architecture must be continuously validated against active, adversarial degradation.

Marabunta embeds a native Chaos Engine (src/chaos/) directly into the daemon. This is not an external testing tool; it is a core subsystem capable of artificially inducing catastrophic physical and cryptographic failures to measure the Swarm's autonomous recovery vectors.

The Implementation: src/chaos/engine.rs

The Chaos Engine forces the execution and transport layers into volatile states, simulating state-actor interference or physical infrastructure collapse.

```
use crate::swarm::types::{NodeId, SwarmRole};
```

```
pub struct ChaosEngine { enabled: bool, fault_probability: f64, }
```

```
impl ChaosEngine { /// Simulates a catastrophic hardware or network failure mid- execution. pub
  async fn inject_fault(&self, current_role: &SwarmRole, target: &NodeId) { if !self.enabled ||
    rand::random::() > self.fault_probability { return; }
```

```
    match rand::random:::<u8>() % 4 {
      0 => {
        // Simulate physical power loss or SIGKILL
```

```
tracing::warn!("CHAOS SIMULATION: Executing  
simulated
```

```
panic. Node dropping offline."); std::process::abort(); }, 1 => { // Simulate a Byzantine Actor  
injecting corrupted gradients tracing::warn!("CHAOS SIMULATION: Mutating WASM linear  
memory to corrupt STARK payload."); self.corrupt_working_memory().await; }, 2 => { // Simulate  
an ISP-level BGP blackhole or submarine cable cut tracing::warn!("CHAOS SIMULATION:  
Dropping all TCP/ QUIC routing tables. Simulating isolation.");  
self.blackhole_network_interfaces().await; }, _ => { // Simulate a highly coordinated Sybil attack  
flooding the DHT tracing::warn!("CHAOS SIMULATION: Spoofing Kademlia XOR metrics to  
saturate local buckets."); self.flood_k_buckets().await; } } }
```

Architectural Analysis: Proving BFT Finality

1. The Data Poisoning Simulation (Case 1): When the Chaos Engine deliberately mutates the WASM linear memory, it forces the node to generate a cryptographically invalid STARK receipt. This validates that the Harpy daemon is functioning correctly on the receiving nodes, confirming that corrupted payloads are instantly slashed and excluded from the IsomorphicStateRing .
2. The Network Isolation Simulation (Case 2): When the engine blackholes the TCP/ QUIC tables, it simulates a kinetic strike on a regional datacenter. The Swarm's recovery is measured by the time (T_{recovery}) it takes the surrounding Kademlia nodes to detect the dropped pings and autonomously re-route the active JobControlLanguage chunks to adjacent latency cohorts.

By compiling the Chaos Engine into the core binary, Cambrian Radiation guarantees that the

mathematical limits of the Marabunta Swarm are constantly stressed, measured, and verified under simulated hostile conditions.

VOLUME 04: THE PHANTOM

OVERLAY (UNIDENTIFIABILITY & ONION ROUTING)

4.0 The Sovereign Egress Diode

Traditional distributed systems rely on the IP-routing layer for data distribution, inherently exposing the network's topology to deep packet inspection (DPI) and traffic analysis by intermediate ISPs or hostile intelligence agencies. Even with TLS encryption, metadata—such as the origin, destination, and timing of packet bursts—remains visible, allowing adversaries to map computational clusters and infer dataset sensitivity.

Marabunta neutralizes traffic analysis by natively embedding the Phantom Overlay, a compact, cryptographic onion-routing protocol specifically optimized for high-throughput computational payloads.

When a regulated institution (e.g., a central bank or genomic lab) deploys a Marabunta node, it functions as an Asymmetric Data Diode. The node binds no listening ports to the public internet, eliminating the possibility of external port-scanning or remote exploitation. It exclusively initiates outbound UDP streams (QUIC) into the global mesh, masked by continuous cover traffic.

4.1 Sphinx: Bit-Level Unidentifiability

The cornerstone of the Phantom Overlay is the Sphinx packet format. Sphinx provides forward secrecy and bit-level unidentifiability, ensuring that as a packet traverses multiple relay nodes, its bitwise representation changes completely at each hop.

The Implementation: `src/swarm/crypto/sphinx.rs`

The following extraction demonstrates the multi-hop nested encryption using x25519-dalek for ephemeral key exchange and ChaCha20 for high-speed stream ciphering.

// Sphinx Packet Nesting: From Exit to Entry for _ in hops.iter().rev() {

```
    let mut key = [0u8; 32];
    rng.fill_bytes(&mut key); // Deriving ephemeral hop key

    let mut nonce = [0u8; 12];
    rng.fill_bytes(&mut nonce);
```

```

// Encapsulate payload layer using ChaCha20
let mut cipher = ChaCha20::new(&key.into(), &nonce.into());
cipher.apply_keystream(&mut current_payload);

// Prepend nonce for the subsequent hop's decryption
let mut next_p = nonce.to_vec();
next_p.extend_from_slice(&current_payload);
current_payload = next_p;

```

```

}

```

Architectural Analysis: Metadata Eradication

1. Ephemeral Handoff: Notice the use of `hops.iter().rev()` . Each layer of the onion is encrypted using a unique, ephemeral symmetric key negotiated via X25519 with each intermediate relay. A relay node (e.g., a consumer laptop in Argentina) only possesses the key to peel its specific layer. It can see the address of the next hop, but it cannot see the ultimate destination, the original source, or the contents of the payload.
2. Fixed Cell Size: Sphinx packets are padded to a fixed size (typically 1300 bytes). This prevents observers from using packet length to distinguish between a tiny command-and-control signal and a massive Cooperative gradient shard.

Egress Diode Backpressure (QUIC Congestion Collapse)

Continuous UDP cover traffic lacks native TCP congestion control. If a Gateway blasts a 10GB tensor over Sphinx UDP to a slow edge node in Argentina, intermediate ISP routers will drop 90% of the packets, crashing the subnet in a self-inflicted DDoS.

Marabunta mitigates this by embedding TCP-Friendly Rate Control (TFRC) and BBR congestion algorithms directly inside the Sphinx QUIC tunnels (`src/phantom/network/ onion.rs`). The Swarm dynamically throttles cryptographic injection rates based on real-time packet-loss telemetry from

the DHT, ensuring high-throughput operations gracefully back off before saturating the physical carrier lines.

4.2 The Continuous Cover Traffic Vector

Even with onion routing, the timing of transmissions (e.g., synchronous weight updates every 4 hours in a federated learning run) creates a statistical "cadence" that signals intelligence agencies can detect.

Marabunta neutralizes this through Continuous Cover Traffic.

1. The White Noise Stream: Marabunta nodes generate a constant, uniform-rate stream of UDP packets (fake traffic) between peers in the DHT.
2. The Payload Injection: When a real computational payload (e.g., a Sphinx-encrypted tensor) needs to be transmitted, it is fragmented and injected seamlessly into this pre-existing white-noise stream.
3. The Result: To a Tier-1 ISP or an intelligence agency tapping an undersea fiber cable, the Marabunta Swarm appears as a flat, meaningless line of entropy. The origin, destination, and exact start/stop times of a scientific or financial sync are mathematically unidentifiable.

Implementation: `src/phantom/network/onion.rs`

```
// Traffic Analysis Resistance: Fixed cell size with random padding pub const CELL_SIZE: usize = 512;
```

```
impl OnionRouter { /// Wrap payload in onion layers (one encryption per hop). fn wrap_onion(&self, keys: &[SymmetricKey], payload: &[u8], rng: &mut impl Rng) -> Result<Vec, OnionError> { let mut data = vec![0u8; 2 + payload.len()]; // Add length prefix and payload...
```

```
    // Encrypt from exit to guard (outermost layer)
    for key in keys.iter().rev() {
        data = key.encrypt(&data, rng);
    }
```

```

        Ok(data)
    }

    /// Add timing jitter to prevent timing analysis.
    async fn add_jitter(&self, rng: &mut impl Rng) {
        let jitter_ms =
            rng.gen_range(0..=self.config.max_jitter_ms);

        tokio::time::sleep(Duration::from_millis(jitter_ms)).await;
    }
}

```

By combining Fixed-Size Cells, Timing Jitter, and Onion Layering, Marabunta provides an

execution environment that remains invisible to the physical infrastructure of the internet. The network is not just encrypted; it is topologically anonymous.

4.3 Absolute Infiltration: DNS-over-HTTPS (DoH) Tunneling

While Sphinx UDP routing provides optimal throughput, totalitarian nation-states (e.g., China, Iran) deploy Deep Packet Inspection (DPI) firewalls capable of executing "default-deny" policies on all unrecognized UDP payloads. If a sovereign actor kinetically severs standard port traffic, the Swarm must maintain command-and-control connectivity.

Marabunta natively embeds Fallback 5 (Last Resort): The DNS-over-HTTPS (DoH) Tunneling Transport.

Implementation: `src/marabunta/transport/doh.rs`

If the marabunta-visor detects that the Sovereign Egress Diode is fully blackholed by an ISP, it autonomously down-shifts the transport protocol.

It encodes the 1300-byte Sphinx computational frames into Base64 strings and fragments them into raw DNS TXT record queries. These queries are routed explicitly through whitelisted, global HTTPS providers (e.g., Google 8.8.8.8 or Cloudflare 1.1.1.1).

```

// DNS TXT limits dictate fragmentation geometry
const DNS_TXT_MAX_BYTES: usize = 255;
const QUERIES_PER_FRAME: usize = FRAME_SIZE.div_ceil(DNS_TXT_MAX_BYTES);

```

```
// Inbound/Outbound Queues structure the Base64 DNS encoded streams pub struct DohTransport
{ codec: FrameCodec, connected: bool, inbound: VecDeque, outbound: VecDeque, }
```

Architectural Analysis: The Un-Censorable Heartbeat

This transport layer operates at a severely constrained bandwidth (approximately ~1 KB/s effective throughput). It cannot be used to synchronize a 350PB Cooperative tensor gradient.

However, it is mathematically sufficient to sustain the minimum viable cryptographic heartbeat

of the Swarm. A node trapped behind a national firewall can use the DoH transport to slowly bleed out execution receipts, authenticate MclManifest command definitions, and maintain its Kademlia routing presence.

To block this traffic, the host nation-state would be forced to permanently block all HTTPS requests to global DNS resolvers, effectively disconnecting their entire populace from the modern internet. The Swarm weaponizes the adversary's reliance on fundamental infrastructure to guarantee its own survival.

4.4 Stigmergic Delay-Tolerant Networking (DTN)

The hyperscaler paradigm assumes constant, low-latency terrestrial fiber connectivity. If an AWS region loses uplink to the internet, all compute halts until the physical connection is repaired.

Marabunta assumes a radically hostile connectivity environment. The Swarm is engineered to operate across deep-space telemetry links, air-gapped secure facilities, and highly volatile intermittent connections (e.g., ships at sea or disconnected IoT edge devices).

Marabunta natively embeds a Stigmergic Delay-Tolerant Networking (DTN) router.

Implementation: src/swarm/transport/stigmergic_dtn.rs

The DTN router mathematically decouples the Kademlia routing plane from the physical transport layer. It operates on a "store-and-forward" cryptographic caching model.

```
// The DTN Router handles massive temporal partitions, bridging air- gapped or deep-space
environments. pub struct DtnRouter { storage: Arc, /// Pending frames waiting for physical
connectivity to a target or intermediate relay. pending_frames: DashMap<NodeId, VecDeque>, }
```

```
impl DtnRouter { /// Commits a cryptographic frame to persistent NVMe storage when the target ///
is physically unreachable (e.g., satellite occultation or physical air-gap). pub async fn
```

```
store_for_forwarding(&self, target: NodeId, frame: Frame) -> Result<(), DtnError> { let  
stored_frame = StoredFrame {
```

```
        frame,  
        stored_at: Instant::now(),  
        expires_at: Instant::now() +
```

```
self.config.max_storage_time, retries: 0, };
```

```
self.pending_frames.entry(target).or_default().push_back(stored_frame); // Persist to local  
BlobStore to survive hard reboots during the partition self.storage.persist_frame(target,  
&stored_frame).await?;
```

```
        Ok(())  
    }
```

```
}
```

Architectural Analysis: Asynchronous Physics

If an edge node (e.g., a drone) enters an area with zero connectivity for two weeks, it does not discard computational payloads. It executes the WASM logic, generates the STARK receipts, and routes the packets to the local DtnRouter .

The node writes the encrypted Sphinx packets to physical NVMe storage. When the node eventually detects an intermittent connection (e.g., passing within range of a LEO satellite or establishing a brief STUN connection via a 3G tower), the DtnRouter autonomously flushes the pending_frames queue into the Kademlia DHT.

The Swarm relies on Epidemic Stigmergy. The packets are passed from node to node, held in storage during outages, and forwarded when links are re-established. The CRDT IsomorphicStateRing is mathematically indifferent to time. It seamlessly merges a tensor gradient computed 14 days ago on an isolated drone with the live global model, neutralizing the hyperscaler assumption of constant connectivity.

VOLUME 05: THE GEOPOLITICAL

MEMBRANE (JURISDICTION FENCING)

5.0 The Obsolescence of the Corporate Firewall

Traditional cloud security relies on physical datacenter perimeters to enforce data residency laws (e.g., GDPR in the European Union, ITAR in the United States, or the Chinese Cybersecurity Law). A developer explicitly provisions an AWS S3 bucket in the eu-central-1 region, trusting that Amazon will not physically replicate the hard drives to an American facility.

This reliance on centralized, physical geography is a structural vulnerability. If an engineer misconfigures a Kubernetes manifest or a Terraform deployment, highly classified or regulated data can instantly leak across sovereign borders, triggering massive federal fines, loss of defense contracts, and diplomatic incidents.

Because the Marabunta Swarm is fundamentally borderless and operates across untrusted, heterogeneous edge hardware, it abandons the concept of the physical firewall entirely. If data can flow anywhere, the network itself must become the security membrane.

Marabunta enforces Jurisdiction Fencing at the mathematical layer. It does not trust a node's IP address. It cryptographically proves the physical location of the execution environment before dispatching a payload.

5.1 Proof-of-Location (PoL) via RTT Triangulation

A malicious node operator in Russia could modify their local marabunta-visor binary to falsely report its country_code as "US", attempting to trick the Swarm into routing classified American computational payloads to their hardware.

To neutralize this attack vector, Marabunta utilizes Round-Trip Time (RTT) Triangulation.

1. The Watchtowers: The Swarm maintains a subset of highly trusted, geographically anchored "Watchtower" nodes (e.g., a Marabunta assimilation node physically bolted inside a US Department of Defense datacenter or an AWS us-east-1 rack).

2. The Ping-Shale: When an unknown edge node claims to reside in the United States, the Watchtower transmits a sub-microsecond UDP ping containing a cryptographic nonce N, recording the transmission timestamp T1.

3. The Physics Constraint: The target node must sign N with its Chrysalis-derived identity key and return it. The Watchtower receives the signature at timestamp T2 and calculates the latency: $RTT = T2 - T1$.

4. Mathematical Verification: The speed of light through fiber optic cables is a hard physical constant (approximately 200,000 kilometers per second). If a node claims to be in Washington D.C., but the Watchtower in Virginia measures an RTT of 140ms, the node is mathematically proven to be physically located in Eastern Europe or Asia.

The math cannot be faked. A node cannot return a cryptographic signature faster than the speed of light allows. If the validation fails, the connection is instantly severed, and the Harpy daemon blacklists the node for cryptographic perjury. The geopolitical membrane remains impermeable.

The Implementation: src/Hardened/jurisdiction.rs

```
use serde::{Serialize, Deserialize};
```

```
#[derive(Clone, Copy, Debug, PartialEq, Eq, PartialOrd, Ord, Hash, Serialize, Deserialize)] pub
enum ZoneClass { Civilian = 0, GovCloud = 1, MilRestricted = 2, MilClassified = 3, }
```

```
#[derive(Clone, Debug, Serialize, Deserialize)] pub struct Jurisdiction { pub country_code: String,
pub min_zone_class: ZoneClass, pub authority_pubkey: Option<Vec>, }
```

```
impl Jurisdiction { /// Evaluates whether a target node mathematically possesses the right to process
/// a specific workload. If it fails, the transport layer drops the TCP/QUIC stream. pub fn
mathematically_validates(&self, node_proof: &NodeGeopoliticalProof) -> bool { // 1. Strict
Border Check
```

```
        if self.country_code != "ANY" && self.country_code !=
```

```
node_proof.country_code { return false; }
```

```

        // 2. Hardware Classification Check
        if node_proof.zone_class < self.min_zone_class {
            return false;
        }

        // 3. Cryptographic Authority Verification
        if let Some(required_key) = &self.authority_pubkey {
            if let Some(node_sig) = &node_proof.authority_signature
            {
                // In production, this verifies the node's

```

certificate was explicitly // signed by the regulatory body using `ed25519_dalek::Verifier`. if `node_sig.len() != 64` { return false; } } else { return false; } }

```

        true
    }
}

```

5.2 Dynamic Governance via Policy DSL

Static geographic constraints are sufficient for basic compliance, but enterprise orchestration requires dynamic, Turing-complete governance.

To achieve this without compromising the deterministic safety of the Swarm, Marabunta embeds a custom Domain-Specific Language (DSL) into the policy engine.

Administrators do not hardcode routing rules into the Rust binary. They write dynamic policies that govern node admission, workload placement, and conflict resolution across the Swarm. The `marabunta-visor` compiles this DSL into an internal Intermediate Representation (IR) for instantaneous evaluation during the `mrbsubmit` deployment phase.

The Implementation: `src/policy/dsl/compiler.rs`

The custom AST (Abstract Syntax Tree) compiler translates human-readable governance rules into executable mathematical constraints.

```

// The Marabunta Policy Engine evaluates the compiled AST
dynamically

```

```
at runtime. pub fn compile_policy_ast(dsl_input: &str) -> Result<PolicyIR, CompilationError> { //
Example DSL Input: // "IF job.priority > 5 AND resource.gpu >= 2 THEN REQUIRE zone.class
== 'MilRestricted'"
```

```
    let tokens = lexer::tokenize(dsl_input)?;
    let ast = parser::parse(tokens)?;

    // The policy is transformed into a deterministic Intermediate
```

```
Representation // that the Swarm routing layer evaluates in < 1ms before executing a payload. let ir
= ir::generate_ir(ast)?;
```

```
    Ok(ir)
```

```
}
```

This architecture allows a corporate compliance officer to inject a new regulatory constraint (e.g., "All financial transactions over \$1M must execute on nodes with Intel SGX Secure Enclaves within the EU") directly into the Swarm's Gossip protocol.

The policy propagates epidemically, and within seconds, an exascale swarm of 100 million or more nodes autonomously adjust their bidding strategies on the MMX spot market to comply with the new mathematical law.

VOLUME 06: THE ASSIMILATION

LAYER (NATIVE INGRESS)

6.0 The Trophic Cascade of Enterprise Migration

The greatest barrier to adopting a planetary-scale, decentralized compute fabric is not network latency; it is the sheer gravitational mass of existing codebases.

If a Tier-1 financial institution relies on 15 million lines of Python and Java tightly coupled to a monolithic PostgreSQL database, or if the Department of Energy relies on 40-year-old Fortran 77 fluid dynamics simulations to model nuclear decay, they cannot simply "migrate" to a peer- to-peer WebAssembly Swarm.

A "lift-and-shift" migration to a novel distributed computing paradigm requires an unjustifiable capital expenditure, decades of rewriting, and introduces catastrophic, systemic operational risks.

Marabunta bypasses the rewrite problem entirely through the deployment of Gateway Assimilators.

These nodes act as technological Transparent Protocol Adapters. They mimic the native wire protocols of existing centralized enterprise infrastructure (e.g., PostgreSQL, S3, OpenAI) and the compiler toolchains of user's scientific compute (Fortran, R, MATLAB).

They silently intercept existing communication streams and source code, shred the requests into distributed wasm32-wasi MapReduce payloads, execute the computation across the global Marabunta Swarm, and return standard, protocol-compliant responses.

The host application—and the scientists or engineers operating it—remains entirely oblivious to the fact that their backend is now a 100-million node biological mesh.

6.1 Enterprise Ingress: The PgWire Compiler

The most structurally complex enterprise Assimilator is the PostgreSQL Gateway. It intercepts native SQL wire traffic and translates it into decentralized WebAssembly in milliseconds.

```
Consider an existing Python application relying heavily on psycopg2
. To migrate this to
```

Marabunta, the engineers change exactly one line of code: the DATABASE_URL connection string, pointing it to a local Marabunta Assimilator.

Implementation: src/api/pgwire/server.rs

```
use pgwire::api::query::{PlaceholderExtendedQueryHandler, SimpleQueryHandler}; use
pgwire::api::results::{DataRowEncoder, FieldFormat, FieldInfo, QueryResponse, Response}; use
sqlparser::dialect::PostgreSqlDialect; use sqlparser::parser::Parser;
```

```
pub struct SwarmPgWireHandler { swarm_client: Arc, }
```

```
#[async_trait::async_trait] impl SimpleQueryHandler for SwarmPgWireHandler { async fn
do_query<'a, C>(&self, _client: &C, query: &'a str) -> PgWireResult<Vec<Response<'a>>> { let
dialect = PostgreSqlDialect {}; let ast = Parser::parse_sql(&dialect, query) .map_err(|e|
PgWireError::UserError(Box::new(e)))?;
```

```
// 2. JIT Compile the AST query structure into a distributed
```

```
Marabunta Task let swarm_task = self.compile_ast_to_task(&ast[0])?;
```

```
// 3. Dispatch the task to the Swarm execution layer.
let swarm_result =
```

```
self.swarm_client.dispatch_task(swarm_task).await .map_err(|_|
PgWireError::IoError(std::io::Error::new(std::io::ErrorKind::Other, "Swarm execution failed")))?;
```

```
// 4. Reconstruct the standard PostgreSQL binary wire
```

```
response let mut results = Vec::new(); for row in swarm_result.rows { let mut encoder =
DataRowEncoder::new(swarm_result.schema.clone()); for field in row {
encoder.append_field(&field)?; } results.push(encoder.finish());
```

```
}
```

```
Ok(vec!
```

```
[Response::Query(QueryResponse::new(swarm_result.schema, results))] } }
```

Architectural Analysis: The Illusion of Centralization

1. The AST Shredder: The Assimilator does not attempt to map SQL to a local relational table. It uses the `sqlparser-rs` crate to break the query down into its fundamental mathematical operations (Filters, Projections, Aggregations).
2. JIT Compilation: The Assimilator translates the rigid SQL logic into an asynchronous Swarm payload, routing the execution to the nearest available worker node possessing the required dataset chunk.
3. Protocol Masking: The reconstruction loop (`DataRowEncoder::new`) ensures that even though the data was processed by 5,000 disparate edge nodes globally, the Python script in New York receives a perfectly formatted PostgreSQL TCP packet. It believes it just queried a massive, centralized Relational Database Service (RDS).

The ACID Transaction Conundrum (Handling Writes)

The `do_query` implementation above demonstrates a SELECT (Read) operation, which is trivially distributable via MapReduce. However, enterprise databases require ACID (Atomicity, Consistency, Isolation, Durability) guarantees for INSERT and UPDATE transactions.

Marabunta bypasses synchronous locking entirely.

When the AST Shredder detects an INSERT or UPDATE , it does not execute a SQL command. It compiles the state mutation into a Conflict-Free Replicated Data Type (CRDT) payload.

1. The Assimilator immediately returns a 200 OK (or the Postgres equivalent `CommandComplete` packet) to the calling application, confirming the transaction was ingested.
2. The payload is injected into the `IsomorphicStateRing` .
3. The global Swarm utilizes commutative and associative mathematics to merge the concurrent mutations asynchronously. If New York adds \$100 and London subtracts \$50, the mathematical outcome is identical regardless of the order the packets arrive at any given node.

The Cryptographic Hash Throughput Collision (Epoch Rollups)

While the CRDT architecture resolves concurrent writes mathematically, a severe performance bottleneck arises at the verification layer. If the Swarm requires a 12ms Cryptographic Hash

verification for every individual SQL INSERT transaction, the global database throughput is

artificially capped to an abysmal ~ 80 Transactions Per Second (TPS).

Marabunta resolves this throughput ceiling via Asynchronous Epoch Rollups ([src/batching/batcher.rs](#)).

The Assimilator coalesces thousands of concurrent mutations into a discrete, time-bound batch (an Epoch). The entire Epoch is compiled into a single WASM payload and executed across the Swarm. The resulting Cryptographic Hash receipt mathematically proves the validity of 10,000 concurrent mutations simultaneously in the same 12ms verification window, elevating the theoretical global TPS limit to over 800,000.

6.2 Heterogeneous Native Ingress (The Global Dependency Resolver)

The architectural philosophy of Marabunta is absolute, unyielding pragmatism: "If a human can use software tools, why can't Marabunta?"

While WebAssembly provides a flawless, secure execution sandbox for custom code (Python, Rust), the reality of global commerce is defined by chaotic, proprietary, undocumented pipelines. The majority of enterprise computation does not run on elegant Fortran scientific models; it runs on fragile, heavily nested Microsoft Excel macros chained to fragmented databases like MongoDB, Oracle, and MariaDB.

Traditional hyperscalers attempt to force these pipelines into pristine, centralized Docker containers. If a corporate analyst needs to run a monthly VBScript macro and sum the results in an Oracle database, IT must provision an expensive Windows Server EC2 instance, pay for an Oracle RDS instance, and write massive, brittle API glue code to connect them (e.g., Apache Airflow or AWS Step Functions).

Marabunta recognizes that the global mesh already possesses these tools natively.

The MCL execution pipeline:

Instead of trying to rewrite Microsoft Excel into wasm32-wasi , Marabunta treats proprietary software as Execution Oracles ([src/switchboard/oracles/](#)).

1. The Job Manifest: An administrator submits a fragmented workflow via Marabunta
Command Language (MCL):
 - Step 1: { execute: "monthly_report.vbs", require_software:

```
["msexcel"] }
```

```
◦ Step 2: { aggregate: "SELECT SUM(revenue)",  
require_software:  
  ["sql_rdbms"] }
```

2. Step 1 (The Excel Execution): The Kademlia DHT queries its Bloom Filters for the

msexcel capability. It locates an idle receptionist's laptop in London running Windows

11. The node receives the CSV, executes the VBScript natively through the Marabunta

Plugin Host, and outputs the raw result matrix.

3. Step 2 (The Multi-Node Aggregation): The London laptop does not possess a database

engine. It gossips the result matrix back to the DHT, requesting the sql_rdbms

capability. The Swarm routes the payload to a 64GB Linux server in Tokyo running

MariaDB (or DB2).

4. The Result: The Tokyo server acts as the Aggregator , ingesting the Excel results via

its native database engine, running the SUM calculation, and returning the final payload to the Gateway.

If a single, high-tier "Aggregator" node happens to possess both an active MS Excel license and a PostgreSQL 15 installation, Marabunta effortlessly routes the entire pipeline to execute locally. If not, the Swarm shatters the workflow, routing the fragments to whatever hardware on Earth natively possesses the required software.

Marabunta functions as a Global Dependency Resolver. It chains together heterogeneous, native, proprietary software across physically distinct machines in real-time, executing chaotic corporate workflows without requiring a single API integration or centralized VM provisioning.

6.3 Scientific Ingress: Fortran & The AST Decoupler

While enterprise banking relies on SQL and proprietary monoliths, the world's most critical scientific and military infrastructure (e.g., NOAA weather forecasting, DoD nuclear stockpile stewardship) relies on millions of lines of heavily aged Fortran.

Marabunta provides native C Foreign Function Interface (FFI) bindings to ingest these models into the decentralized wasm32-wasi execution sandbox.

The Fortran 77 Nightmare

Modern Fortran (F90, F95, F03) is natively supported by the Marabunta CLI. It utilizes the ISO_C_BINDING module, allowing the Marabunta LLVM toolchain (via Flang/Clang) to compile the source code directly into deterministic WASM payloads seamlessly.

However, Fortran 77 does not possess ISO_C_BINDING . It relies on implicit typing and, catastrophically, COMMON blocks (global shared memory). You cannot simply compile a

```
stateful, shared-memory architecture into a distributed, stateless
WASM node without breaking
```

the mathematics.

Implementation: src/bin/assimilator.rs

Marabunta resolves this through Compiler-Grade AST Decoupling.

```
pub struct FortranAnalyzer;
```

```
impl FortranAnalyzer { /// Compiler-grade AST Decoupling of Fortran 77 pub fn
decouple_ast_and_refactor(source_code: &str) -> Result<String, AssimilationError> { // 1.
Generate the Abstract Syntax Tree via Tree-sitter let tree =
Self::parse_f77_syntax_tree(source_code)?;
```

```
    // 2. Identify all shared memory violations (COMMON blocks,
```

```
EQUIVALENCE) let violations = Self::identify_stateful_memory_blocks(&tree);
```

```
    // 3. Algorithmically refactor global state into discrete,
```

```
passed variables // compatible with stateless WASM execution environments. let refactored_source
= Self::rewrite_ast_for_wasm(tree, violations)?;
```

```
    Ok(refactored_source)
}
```

}

The FortranAnalyzer intercepts the legacy .f77 code. It uses Tree-sitter to algorithmically decouple the shared memory COMMON blocks into discrete, stateless variables, and JIT-compiles the modernized AST into a deterministic wasm32-wasi payload.

By supporting R (Rcpp), MATLAB, C++, and automated F77 modernization alongside native MS Excel and Oracle execution, Marabunta positions itself not just as a cloud alternative, but as the ultimate, frictionless upgrade path for global existing infrastructure.

VOLUME VII: NEUROMANCER (THE

PREDATOR DAEMONS)

7.0 The Architecture of Autonomous Eradication

A planetary swarm without a native, highly aggressive immune system will inevitably collapse. If 100 million or more (e.g., 15 billion) nodes are connected globally, thousands or millions of them will be compromised, malfunctioning, or actively malicious at any given second.

Traditional cloud infrastructure relies on reactive defense postures: Security Operations Centers (SOCs), Splunk dashboards, heuristic alerting, and human administrators writing firewall rules or banning IP ranges.

Marabunta abandons the human-in-the-loop paradigm. It deploys Neuromancer, an autonomous, mathematically ruthless immune system composed of independent "Predator Daemons."

These Rust-based asynchronous event loops continuously traverse the XOR metric space of the Distributed Hash Table (DHT), enforcing mathematical laws and eliminating topological anomalies without human intervention. They do not accept user input. They do not have a REST API. They only listen to the internal telemetry of the node and the cryptographic signatures of the DHT gossip.

7.1 The Harpy Eagle : Cryptographic Hash Execution Validation

The most lethal daemon in the Neuromancer suite is the Harpy Eagle . Its sole purpose is to hunt for mathematical perjury in the execution trace of distributed payloads.

When a Marabunta node completes a computational task (e.g., a Cooperative tensor gradient update or a MapReduce aggregation), it cannot simply broadcast the result. It must attach a Zero-Knowledge STARK receipt—a dense polynomial commitment proving that it executed the exact wasm32-wasi payload required, without exposing the underlying data or algorithms.

Implementation: src/swarm/neuromancer/harpy.rs

```
use std::sync::Arc; use tokio::sync::mpsc::Receiver;
```

```
pub struct HarpyEagle {  
    zkp_engine: Arc<ZkpEngine>,  
    transport: Arc<TransportHandle>,  
    inbound_proofs: Receiver<(NodeId, ExecutionProof)>,  

```

```
}
```

```
impl HarpyEagle { pub async fn hunting_loop(&mut self) { while let Some((suspect_node, proof))  
= self.inbound_proofs.recv().await { // The mathematical guillotine. // Evaluates the FRI  
polynomial commitment in exactly 12 milliseconds. let is_valid =  
self.zkp_engine.verify_proof(&proof);
```

```
    if !is_valid {  
        // 1. Generate the cryptographic Slasher Witness  
        let witness = SwarmMessage::SlasherWitness {  
            target: suspect_node,  
            violation_type:
```

```
"INVALID_STARK_TRACE".to_string(), proof_hash: proof.public_inputs[0], };
```

```
        // 2. Broadcast the execution order to the DHT.  
        self.transport.broadcast_epidemic(witness).await;  
  
        // 3. Sever local TCP/QUIC connections  
        immediately.  
            self.transport.drop_peer(&suspect_node).await;  
        }  
    }  
}
```

```
}
```

As packets hit the SwarmTransport layer, they are piped into the Harpy Eagle queue. The daemon executes the Fast Reed-Solomon Interactive Oracle Proofs of Proximity (FRI) verification.

If the trace hash deviates by a single bit from the mathematically expected state transition curve, the Harpy Eagle immediately generates a SlasherWitness payload, broadcasts it to the global

network, and physically drops the TCP connection to the offending node. The offending node is excommunicated from the Swarm in less than 15 milliseconds.

7.2 The Wild Dogs : Topological Sybil Eradication

The Wild Dogs daemon analyzes the density of the Kademlia routing buckets. If it detects a statistically impossible clustering of Node IDs in a specific 2160 address space, it flags a potential state-actor attempting an Eclipse route hijack.

Implementation: src/swarm/neuromancer/wild_dogs.rs

```
pub struct WildDogs { threshold_density: f64, address_space_usage: HashMap<u8, usize>, }

impl WildDogs { pub fn detect_topological_anomaly(&mut self, table: &RoutingTable) -> Vec {
let mut suspicious_nodes = Vec::new();
```

```
    for (bucket_idx, bucket) in table.buckets.iter().enumerate()
```

```
{ let density = bucket.len() as f64 / 20.0;
```

```
        if density > self.threshold_density {
            let entropy =
self.calculate_bucket_entropy(bucket);
            if entropy < 0.4 {
                // Low entropy indicates a single actor
```

```
generating identities for peer in bucket { suspicious_nodes.push(peer.id); } } } suspicious_nodes
    } }
```

By applying Shannon Entropy functions to the bit-patterns of the NodeIDs within the buckets, Wild Dogs differentiates between organic network growth and coordinated, malicious identity generation (Sybil nodes attempting to surround a target). It preemptively slashes the connections and broadcasts a PRUNE message to the DHT.

7.3 Auxiliary Predator Daemons

The Neuromancer ecosystem includes auxiliary daemons dedicated to optimizing Swarm topology, data integrity, and execution scheduling over time.

Viper (Dynamic Traffic Obfuscation)

The Viper daemon constantly rotates the QUIC connection IDs and UDP port bindings for the Sovereign Egress Diode. It ensures that deep packet inspection (DPI) firewalls cannot correlate traffic flows between specific Swarm nodes over extended periods, frustrating sustained traffic analysis and commercial espionage attacks.

Crocodile (Orphaned Data Scavenging)

As the network scales or shrinks, the mathematical mapping of data shards to their required Kademlia neighbors shifts. The Crocodile daemon constantly scavenges the local BlobStore . It identifies and relocates orphaned shards to new topological neighbors, ensuring the Reed-Solomon parity matrix remains intact despite extreme network churn.

Lazarus (Cryptographic Resurrection)

When an edge node crashes or suffers a catastrophic power failure, the .mrb-dump Flight Data Recorder is isolated. The Lazarus daemon scans the DHT for nodes that have mathematically proven their prior identity but have gone silent. It resurrects their execution state on a surrogate node, pulling the execution logs and resuming the deterministic WASM payload from the exact instruction pointer where the failure occurred.

Wintermute (The Orchestrator)

While the predators cull the weak and the malicious, the Wintermute daemon acts as the central task scheduler. It interfaces with the NeuromancerBus to manage the PendingTask queue, tracking the exact retry limits (`MAX_RETRIES: u32 = 3`). If Lazarus fails to resurrect a payload after three attempts due to insurmountable topological degradation, Wintermute definitively marks the task as failed, triggering the requisite SLA (Service Level Agreement) penalties defined in the JCL.

VOLUME 08: THE FLUID TOPOLOGY

(MULTIPLEXED ROLES & RECURSIVE DELEGATION)

8.0 The Eradication of the Hardware Caste

The initial architecture of decentralized networks often relies on static hardware castes—categorizing a 4GB laptop as a "dumb worker" and a 64GB server as a "smart router." This hierarchical, centralized thinking is a catastrophic waste of thermodynamic potential.

Marabunta formally deprecates the concept of static hardware classes. A node's physical CPU cores or RAM do not dictate its destiny. The Swarm is a Fluid Topology driven entirely by the MMX Spot Market.

8.1 The Multiplexed Hypervisor

A single Marabunta Node (the marabunta-visor binary) is not a single actor. It is a multiplexed hypervisor running concurrent asynchronous Tokio tasks.

At timestamp $T=100$, a single 16GB gaming PC in London can simultaneously execute: * Task A (SwarmRole::Aggregator): Waiting for 1,000 other nodes to return shards of a Monte Carlo climate simulation. * Task B (SwarmRole::Worker): Computing chunk #452 of its own Monte Carlo simulation for Task A. * Task C (SwarmRole::Relay): Forwarding encrypted Sphinx UDP packets for a completely unrelated legacy batch job originating in Tokyo. * Task D (SwarmRole::Storage): Pinning a 50MB Reed-Solomon shard of an enterprise DB2 database to its NVMe drive.

There is no rigid hierarchy. There is only a fluid, hyper-concurrent matrix of available CPU cycles, RAM, and network bandwidth, shifting roles millisecond by millisecond.

8.2 Recursive Delegation (The Sub-Contractor Mesh)

The apex of decentralized economics is Micro-Service Sub-Contracting over the Kademlia DHT. A node can orchestrate compute that it itself is simultaneously participating in, breaking recursion through cryptographic task isolation.

1. The Master Contract: Node 1 (A Mobile Phone) receives a \$10 MMX contract to render a 3D frame.
2. The First Delegation: Node 1 realizes it lacks the GPU. It adopts

the Aggregator
role and sub-contracts the heavy matrix math to Node 2 (A Gaming PC) for \$8 MMX,
capturing \$2 for routing.

3. The Second Delegation: Node 2 accepts the \$8 contract. But it is currently busy mining Chrysalis PoW, so it sub-contracts the math to Node 3 (An AWS Spot Instance) for \$6 MMX.

4. The Recursive Loop (The Execution Oracle): Node 3 begins compiling the WebGL
shader, but it needs an execution oracle for a specific collision physics formula. It
broadcasts a Capability lookup to the Kademlia DHT.

5. The Fulfillment: Node 1 (The Mobile Phone) happens to have that specific physics
library cached in RAM. Node 3 sub-contracts the physics formula back to Node 1 for \$1 MMX.

Why doesn't the recursion crash the network? Because Kademlia nodes are blind to the "Master Contract." Node 3 does not know Node 1 is the original requester of the 3D frame. Node 3 only knows that Node 1 offered to solve a 5-millisecond physics equation for \$1 MMX.

The recursion is broken by the deterministic hash of the specific, isolated TaskID . Every node is simply fulfilling strict, stateless thermodynamic sub-contracts. The global topology is irrelevant; the localized thermodynamic transaction is absolute.

VOLUME 09: THE DSL COMPILER &

DECENTRALIZED GOVERNANCE

9.0 The Limits of Static Configuration

Enterprise orchestration systems traditionally rely on static configuration files (e.g., YAML, JSON) to define cluster behavior. If an administrator wants to change how workloads are prioritized, they must modify the configuration, push it through a CI/CD pipeline, and restart the orchestrator daemons.

In a decentralized swarm spanning 100 million or more nodes, rolling restarts are impossible. Furthermore, hardcoded logic cannot adapt to the infinite edge-cases of a multi-tenant, planetary-scale network where competing institutions (banks, research labs, consumer hardware) constantly bid for the same silicon.

To solve this, Marabunta embeds a bespoke Policy Domain-Specific Language (DSL) directly into the rust binary.

Administrators do not change configuration files; they broadcast mathematical logic. The DSL allows operators to write dynamic, Turing-complete governance constraints that propagate epidemically across the Swarm and are evaluated locally by every node in sub-millisecond execution windows.

9.1 The Custom DSL Compiler

The Marabunta Policy Engine is not a simple regex parser. It features a complete compiler frontend—including a Lexer, Parser, and Abstract Syntax Tree (AST) generator—written in pure Rust.

The Implementation: `src/policy/dsl/parser.rs`

The DSL uses a declarative condition `=> effect` syntax. When a policy is broadcast, the local node compiles it into an internal Intermediate Representation (IR).

// Example DSL Input: `// job.priority > 5 AND resource.gpu >= 2 => require zone.class ==`

```
'MilRestricted'
```

```
pub fn parse(tokens: Vec) -> Result<AstNode, ParseError> { let mut parser = Parser::new(tokens);
  parser.parse_policy() }
```

```
impl Parser { fn parse_condition(&mut self) -> Result<Expression, ParseError> { // Recursive
descent parsing for logical operators (AND, OR, NOT) // and comparison operators (==, >, <, IN)
let left = self.parse_comparison()?;
```

```
    if self.match_token(TokenType::And) {
        let right = self.parse_condition()?;
        Ok(Expression::LogicalAnd(Box::new(left),
```

```
Box::new(right))) } else { Ok(left) } } }
```

Architectural Analysis: JIT Policy Evaluation

When a developer submits a wasm32-wasi payload via mrdb submit , every node evaluating the bid runs the compiled Policy IR. * Zero-Cost Abstraction: Because the DSL is compiled to a binary IR when it first hits the node, evaluating the rule against an incoming job takes less than 1 microsecond. * Dynamic Enforcement: An administrator can instantly push a policy: `node.thermal_load > 85 => block job.type == 'training'` . Within seconds, the entire global Swarm will autonomously reject heavy AI training workloads on hot nodes, completely bypassing the need for a central orchestrator to micromanage placement.

9.2 The Authority Chain (Multi-Tenant Governance)

If multiple organizations share the Swarm, policies will inevitably conflict.

- Bank A broadcasts a policy: `allow node.country == 'US'` .
- Bank B broadcasts a policy: `deny node.country == 'US'` .

Marabunta resolves these conflicts using Cryptographic Authority Chains.

The Implementation: `src/governance/authority.rs`

```
pub struct AuthorityChain { /// The principal whose authority is being traced (e.g., Bank A Admin).
pub principal: PrincipalId, /// The specific domain of control (e.g., 'placement.geographic'). pub
domain: String, /// The cryptographic sequence of delegation proving authority. pub granted_via:
Vec, /// The effective priority (weight) of this specific chain. pub effective_priority: Option, }

pub struct AuthorityChecker { principals: Arc<RwLock<HashMap<PrincipalId, Principal>>>,
delegations: Arc<RwLock<Vec>>>, }
```

Architectural Analysis: Resolving the Byzantine Conflict

1. Cryptographic Principal IDs: Every policy is signed by a PrincipalId (derived from an ed25519 keypair).
2. Domain Delegation: Authority is not absolute. The Genesis node can delegate the placement.geographic domain to the EU Commission's Principal ID, while delegating the economic.bidding domain to the Federal Reserve's Principal ID.
3. Conflict Resolution: When two policies conflict, the AuthorityChecker traverses the granted_via graph. It evaluates the cryptographic signatures up the chain to the Root. The policy signed by the Principal with the highest effective_priority for that specific domain mathematically wins the conflict.

This enables Decentralized Multi-Tenancy. A single global Swarm can securely host the workloads of rival corporations or adversarial nation-states, mathematically guaranteeing that one tenant cannot override the hardware placement or economic policies of another.

VOLUME 10: RUNTIME

EXTENSIBILITY & MCL REFERENCE

10.0 The Extensible Swarm

A planetary-scale infrastructure cannot be a monolithic, immutable black box. To support the diverse requirements of sovereign states, global banks, and edge consumers, Marabunta is architected as a modular substrate.

The system utilizes a high-performance WASM-based Plugin Architecture. This allows administrators to inject custom logic into the marabunta-visor daemon at runtime—without recompiling the binary or restarting the node.

10.1 The Plugin Architecture

Marabunta provides clean, opaque API surfaces for third-party extensions across five critical domains: 1. Transport Plugins: Define custom networking protocols (e.g., Satellite, LoRa, or proprietary radio links). 2. Storage Plugins: Integrate new backend persistence layers (e.g., IPFS, Arweave, or localized NVMe arrays). 3. PubSub Plugins: Inject custom telemetry brokers for real-time monitoring. 4. Scatter/Gather Plugins: Define novel MapReduce partitioning strategies. 5. Migration Plugins: Control how WASM memory states are transferred between nodes during preemption.

Implementation: `src/plugin/mod.rs`

The PluginHost orchestrator manages the full lifecycle of these modules: spawning isolated processes, establishing secure data channels, and monitoring health.

```
pub struct PluginHost { /// Registry of all active and standby plugins. registry: Arc, /// Lifecycle
state machine for each plugin process. lifecycles: RwLock<HashMap<PluginId,
PluginLifecycle>>, }
```

```
impl PluginHost {
```

```
    /// Dynamically loads a signed WASM plugin into the Swarm.
    pub async fn load_plugin(&self, binary: Vec<u8>, config:
```

```
PluginConfig) -> Result<(), PluginError> { // 1. Verify the cryptographic signature of the plugin. //
2. Instantiate the isolated WASM sandbox for the plugin. // 3. Register the plugin's exported
capabilities into the Swarm's trait map. Ok(()) } }
```

This extensibility ensures that Marabunta remains future-proof. If a superior encryption algorithm or a more efficient transport protocol is developed, it can be deployed epidemically across the Swarm as a plugin.

The Plugin Security Model (Capability-Based Isolation)

Hot-swapping executable plugins at runtime introduces severe vector risks. A malicious or poorly written storage plugin could attempt a directory traversal attack (`../../etc/ shadow`) on the host edge node.

Marabunta secures the runtime via WASI-CAP (Capability-Based Security). Plugins are not granted blanket host access. When a transport plugin is initialized, the PluginHost binds it to an exclusive, randomized port matrix. When a storage plugin is loaded, it is passed a virtualized, chroot-jailed block device descriptor. The plugin physically cannot address memory or filesystem paths outside its explicitly granted capabilities.

10.2 Marabunta Command Language (MCL)

To interact with this modular infrastructure, developers and DevOps engineers utilize the Marabunta Command Language (MCL).

MCL is a formal, deterministic markup (typically expressed in YAML or JSON) that codifies the physical, economic, and geopolitical requirements of a computational workload. It serves as the "Instruction Manual" for the biological routing layer.

The Formal Specification: JobSubmissionRequest

The following is the high-level schema of an MCL manifest, demonstrating the precision with which a user can define their execution physics.

```
pub struct MclManifest {
  pub name: String,
  pub script_type: ScriptType, // e.g., "Python", "Rust", "Shell",
```

```
"Cooperative" pub script: String, pub input_blobs: Vec, pub chunk_strategy: ChunkStrategy, // e.g.,
"MapReduce", "Single", "Fractal" pub requirements: JobRequirements, pub energy_budget:
Option, pub priority: u8, // 0-255 }
```

10.3 The JobRequirements Block

The requirements block allows for granular hardware and geographic targeting: * min_vram_gb : Minimum GPU memory required. * require_tee : Boolean flag for Intel SGX/AMD SEV enforcement. * allowed_zones : A list of ISO country codes (e.g., ["US", "CH"]) enforced via PoL RTT triangulation. * max_rtt_ms : Maximum permissible network latency for the aggregation quorum.

10.4 The EnergyBudget and MMX Bidding

MCL introduces Thermodynamic Bidding. Instead of a fixed cost, the user defines their maximum spend in MMX tokens and their bidding_strategy . * Urgent: Bids at the 90th percentile of the current spot market to ensure immediate preemption of other workloads. * Lazy: Bids only when energy prices go negative (e.g., during solar peaks in Texas), ensuring the lowest possible execution cost.

MCL is not merely a configuration format; it is a mathematical contract. Once signed and submitted to a Gateway Assimilator, the MCL manifest is immutable, providing the ZK- STARK pipeline with the baseline constraints required to verify the integrity of the result.

VOLUME 11: THE PROGRAMMATIC

CONTROL PLANE (API & ROUTING MATRIX)

11.0 The Death of the "Fire and Forget" Batch Job

In traditional High-Performance Computing (HPC), a researcher submits a payload to a job scheduler (e.g., Slurm) and waits. If the code contains a syntax error, a missing dependency, or a mathematically unsolvable equation, the system may run for 12 hours on a 10,000-node cluster before failing. The researcher is billed for the wasted compute time.

Marabunta eliminates this inefficiency through the Programmatic Control Plane.

The Gateway Assimilators do not blindly accept payloads. They provide a synchronous, deterministic API that performs localized Dry-Run Compilation and Mathematical Classification before injecting the payload into the planetary Swarm.

11.1 The Switchboard Routing Matrix

When a complex mathematical expression or workflow is submitted to the API, it is routed through the Switchboard.

The Switchboard does not rely on a single execution backend. It utilizes a multi-oracle routing architecture. If an expression is submitted in LaTeX or plain text, the ingest module parses it into an AST, extracts the symbols, and attempts to resolve it using the primary oracle.

If the primary oracle fails (e.g., due to an unsolvable symbolic constraint), the payload does not crash. It gracefully degrades through a mathematically defined 7-Level Failure Hierarchy.

Implementation: `src/switchboard/failure.rs`

```
pub enum FailureLevel { /// Total success, all execution backends responded correctly L0, ///  
    Partial success, primary backend succeeded but secondary verification failed
```

```
        L1,  
        /// Fallback used, primary failed but fallback oracle succeeded  
        L2,  
        /// Degraded result, only a partial computational answer is
```

mathematically available L3, /// Classification only, execution failed but the expression was structurally classified L4, /// Input accepted, parsing failed but the raw payload was stored for asynchronous retry L5, /// Total failure, the expression is mathematically invalid L6, }

```
impl FailureLevel { /// Returns true if this failure level is considered acceptable. /// L0-L3 are acceptable, L4-L6 require escalation or user intervention. pub fn is_acceptable(&self) -> bool { matches!(self, Self::L0 | Self::L1 | Self::L2 | Self::L3) } }
```

This matrix guarantees that an enterprise user paying for compute is never billed for an L6 catastrophic failure, because the Switchboard traps and rejects the payload synchronously at the Gateway before it ever hits the MMX spot market.

11.2 The mrb submit Pipeline

The standard interaction model for an enterprise developer is the Marabunta Command Line Interface (mrb).

When a developer executes `$ mrb submit --manifest aml_job.mcl`, the CLI initiates a highly orchestrated sequence with the ApiServer operating on the Gateway Assimilator.

1. Authentication & RBAC: The Gateway validates the developer's WebAuthn PKI token, ensuring they possess the submit_jobs permission within the specific Jurisdiction defined in the MCL manifest.

2. Synchronous AST Dry-Run: The Gateway compiles the Python/Rust payload into wasm32-wasi . It verifies the memory bounds and ensures all imported host functions are safely stubbed by the MantisJournal . If the code contains an infinite loop

detectable via static analysis, the deployment is rejected with an HTTP 400 Bad Request .

3. Thermodynamic Escrow Lock: The Gateway evaluates the EnergyBudget block of the MCL manifest. It calculates the projected cost based on the current global MMX spot market pricing and locks the required funds in a

```
cryptographic escrow smart  
contract.
```

4. The Epidemic Handoff: Only after the payload achieves an L0 or L1 Switchboard

```
verification status does the API return an HTTP 201 Created  
response to the  
developer. The Gateway then injects the WASM binary and the MCL  
constraints into the  
Kademlia DHT.
```

From the developer's perspective, the API response is instantaneous and familiar. Behind the Gateway, the payload has transitioned into an unstoppable biological virus, propagating across 100 million or more (e.g., 15 billion) nodes via Plumbtree gossip.

11.3 Enterprise Integration (Federated Identity & Asynchronous Event Streams)

While the underlying Swarm operates in a zero-trust, cryptographic vacuum, requiring enterprise developers to manually manage ed25519 keypairs to interact with the Gateway Assimilator introduces catastrophic usability friction.

Marabunta bridges this gap by natively supporting legacy enterprise IT frameworks without compromising the cryptographic integrity of the mesh.

Federated Identity (OIDC/SAML)

The Gateway Assimilator is not a silo. It natively integrates with centralized enterprise Identity Providers (IdP) such as Okta, Microsoft Entra ID, or Ping Identity via the AuthLayer module (`src/swarm/auth.rs`).

When a developer executes `$ mrb submit --manifest aml_job.mcl` , they do not need to provide a raw private key. They authenticate via an OpenID Connect (OIDC) JWT token. The Gateway validates the cryptographic signature of the IdP, maps the developer's corporate group (e.g., `JPMorgan_Data_Science_L3`) to the corresponding Marabunta Jurisdiction permissions, and transparently signs the MCL manifest on their behalf using the Gateway's HSM (Hardware Security Module).

The Swarm remains cryptographically verified, but the enterprise maintains standard SSO (Single Sign-On) lifecycle management over its engineers.

Asynchronous Event Streams (Webhooks)

A 100-million node network does not execute linearly. A massive MapReduce aggregation or a 350PB Cooperative gradient sync may take minutes or hours to clear the Kademlia DHT. If an enterprise CI/CD pipeline or SIEM (Security Information and Event Management) dashboard is forced to continuously poll the Gateway API for status updates, the resulting traffic storm will degrade the Control Plane.

Marabunta solves this via Asynchronous Webhooks (`src/swarm/webhooks.rs`).

An administrator can subscribe a corporate endpoint (e.g., a Splunk ingestor or a PagerDuty listener) to specific Swarm events.

When the Harpy daemon mathematically verifies a Cryptographic Hash trace, or when the IsomorphicStateRing achieves global consensus on a CRDT merger, the Gateway immediately pushes a secure HTTPS POST request containing the event metadata and the cryptographic receipt directly to the subscribed corporate endpoint.

The enterprise is updated instantaneously, without ever polling the network, allowing Marabunta to function as a seamless, event-driven coprocessor for existing IT architectures.

The Dashboard API

Once deployed, the developer does not SSH into an edge node to view logs. The Control Plane API exposes aggregate telemetry while preserving the anonymity of the Phantom Overlay.

Get the latency heatmap for a specific distributed training run

\$ curl http://localhost:8080/api/jobs/wp_aml_syndicate_99/latency

Filter active nodes by thermodynamic region

```
$ curl "http://localhost:8080/api/nodes?region=us-east-1&status=online"
```

The infrastructure endpoints provide full detail on public "Aggregator" nodes, while the "Ephemeral" node telemetry is heavily obfuscated and aggregated to protect the physical locations of consumer hardware participating in the Swarm.

VOLUME 12: THE PANOPTICON

OBSERVATORY (WEBGL WAR ROOM)

12.0 The Geometry of the Superorganism

A centralized architecture operates linearly. A planetary peer-to-peer (P2P) swarm is a biological superorganism. Relying on text-based logs, 2D Grafana dashboards, or Kibana traces is fundamentally insufficient for comprehending the real-time, non-linear topology of a 10- million node network.

The Marabunta Control Plane abandons traditional dashboarding in favor of a bespoke WebGL/ Three.js frontend—The Panopticon Observatory.

This interface visualizes Kademlia XOR routing distances, epidemic gossip propagation, and energy-based market arbitrage in a fully interactive, 3D spatial environment rendered at 60 FPS directly within a web browser.

12.1 The Dimensional Hierarchy (MegaCubes)

To prevent browser memory exhaustion when rendering millions of active, gossiping edge nodes (Ephemeral), the Panopticon utilizes a dynamic, real-time clustering engine (ClusterEngine.ts) based on the "MegaCube" paradigm.

Nodes are not mapped geographically on a 2D map of Earth. Physical geography is irrelevant in a purely XOR-based routing space. They are clustered logically by their topological role and current cryptographic state.

Implementation: src/web_ui/static/ClusterEngine.ts

Rendering 10,000 individual sphere geometries would require 10,000 WebGL draw calls, instantly crashing the client GPU. The frontend circumvents this using InstancedMesh architectures.

It renders a single base sphere geometry and updates the translation matrices (position, scale, and color) of 10,000 instances in a single draw call.

```
import * as THREE from 'three';
```

```
export class ClusterEngine { private instancedMesh: THREE.InstancedMesh; private dummy:  
THREE.Object3D; private color: THREE.Color;
```

```

constructor(maxNodes: number = 1000000) {
  const geometry = new THREE.IcosahedronGeometry(0.5, 1);
  const material = new THREE.MeshBasicMaterial({
    color: 0xffffffff, transparent: true, opacity: 0.85
  });

  this.instancedMesh = new THREE.InstancedMesh(geometry,

```

material, maxNodes);

this.instancedMesh.instanceMatrix.setUsage(THREE.DynamicDrawUsage); this.dummy = new THREE.Object3D(); this.color = new THREE.Color(); }

```

public updateSwarmTopology(telemetryStream: Float32Array,

```

nodeCount: number) { for (let i = 0; i < nodeCount; i++) { const offset = i * 7; // [x, y, z, r, g, b, scale]

```

    this.dummy.position.set(
      telemetryStream[offset],
      telemetryStream[offset + 1],
      telemetryStream[offset + 2]
    );

    const scale = telemetryStream[offset + 6];
    this.dummy.scale.set(scale, scale, scale);
    this.dummy.updateMatrix();

    this.instancedMesh.setMatrixAt(i, this.dummy.matrix);

    this.color.setRGB(
      telemetryStream[offset + 3],
      telemetryStream[offset + 4],
      telemetryStream[offset + 5]
    );
    this.instancedMesh.setColorAt(i, this.color);

```

```

    }

    this.instancedMesh.instanceMatrix.needsUpdate = true;

    this.instancedMesh.instanceColor.needsUpdate = true;
}
}

```

The WebGL Telemetry Protocol

The backend Gateway Assimilator acts as a WebSocket multiplexer. It subscribes to the DHT SwarmMessage::Telemetry gossip, aggregates the states of thousands of nodes into a single, highly compressed binary Float32Array, and pushes it to the WebGL client. This eliminates JSON parsing overhead, allowing smooth 3D rendering of a continent-scale distributed training run on a standard laptop.

Telemetry Anonymization & The Observer Effect

A critical architectural paradox arises: If 10 million edge nodes are streaming real-time telemetry (CPU temperature, network latency, MMX bid status) to a centralized WebGL dashboard, does this not completely violate the operational security of the Phantom Overlay?

If an intelligence agency intercepts the telemetry stream, could they not de-anonymize the Swarm?

Marabunta resolves this through Aggregated Oblivious Telemetry. 1. Nodes do not transmit telemetry directly to the Gateway Assimilator hosting the Panopticon UI. 2. Telemetry is pushed to the local Kademia DHT neighbors. The local \$5ms\$ Latency Cohort aggregator compiles the telemetry of its 10,000 neighbors into a single, obfuscated statistical summary (e.g., "Cohort 0x4F: 80% Utilization, Avg Temp 65C"). 3. The individual NodeId or physical IP address of a specific smartphone or laptop is stripped at the Tier-1 aggregation layer. 4. The Gateway Assimilator only receives these anonymized, aggregated cohort statistics.

When the ClusterEngine.ts renders 1.2 million NodeSphere instances on the screen, it is rendering a mathematically accurate statistical representation of the Swarm's physics, not a literal 1:1 mapping of physical IP addresses. The observer can see the thermodynamic flow of the system, but the privacy of the individual edge node remains cryptographically verified.

12.2 Visualizing Swarm Physics

The Observatory doesn't just display static state; it visualizes the physical motion of the algorithms.

Topological Mode (Kademlia & Roles)

1. Gateways & Aggregators: High-tier nodes float at the top of the Z-axis. Their color intensity (cyan to deep blue) represents their current "Elo" reputation and bandwidth saturation.
2. Ephemeral Cubes: Standard worker nodes are aggregated into massive, slowly rotating 3D cubes at the bottom of the Z-axis. Zooming into a specific MegaCube triggers the `DecomposeEngine.ts`, dissolving the cube into thousands of individual `NodeSphere` meshes and revealing the underlying Ping-Shale connections.
3. The Graveyard: Nodes that have been cryptographically severed by the Harpy daemon or killed due to thermal throttling sink to the absolute floor of the scene, rendered as red, translucent "Ghosts" fading out of the DHT.

Epidemic Gossip Arcs (Plumtree Visualization)

When a new workload is submitted, the 3D scene renders curved, glowing `CrossCubeArcs`. These lasers branch exponentially across the space, visualizing exactly how the mathematical Pheromone gradient spreads outward from the injection point in $O(\log N)$ time.

Thermodynamic Mode (MMX Spot Market Arbitrage)

If an administrator executes a Context Switch from Topology Mode to Economic Mode, the nodes physically reform their clusters based on their bid/ask spread on the MMX spot market. The background grid shifts color based on the global thermodynamic cost of compute.

The administrator can visually identify where compute is currently cheapest (deep blue "cold" zones) and watch computational WASM payloads dynamically gravitate toward those geographic regions.

12.3 The Cryptographic Command Surface

The Observatory is not a read-only monitoring tool. Because the UI is authenticated as a Super-Peer (via HSM or WebAuthn PKI), an Administrator possesses bi-directional God Mode capabilities over the Swarm.

1. Visual BGP Blackholing: If an Administrator visually observes a specific Sub-Cube of nodes turning aggressively red (indicating high latency, thermal saturation, or a localized Sybil attack), they can right-click the 3D cube and select [Quarantine Subnet] .
2. Epidemic Revocation: The UI immediately signs a topological Cryptographic Subnet Quarantine and injects it into the DHT. The Admin watches the connections (GossipEdges) physically snap and dissolve in real-time as the rest of the Swarm isolates the quarantined nodes.
3. Time-Travel Scrubbing: Because the Swarm state is heavily serialized locally, the UI features a TimeTravelScrubber slider. If a systemic cascade failure occurred at 2:00 AM, the Admin drags the slider back in time. The 3D geometry rewinds, mathematically rebuilding the exact state of the network at that specific millisecond for forensic visual inspection.

VOLUME 13: FORMAL

VERIFICATION & Cryptographic Hash CONSTRAINTS

13.0 The Obsolescence of Institutional Trust

Institutional trust is a liability. Relying on paper treaties, corporate Service Level Agreements (SLAs), or the physical perimeter security of a hyperscaler datacenter assumes that all actors will behave honestly when millions of dollars or geopolitical advantages are at stake.

In a decentralized swarm spanning a planetary fabric of 100 million or more heterogeneous nodes, you cannot trust the hardware, and you cannot trust the operator.

Mathematical truth is the only sustainable substrate for global interaction.

Marabunta abandons trust entirely. It relies exclusively on cryptographic verification, deterministic execution paths, and zero-knowledge proofs to enforce the integrity of every computation executed across the Swarm.

13.1 The Cryptographic Hash Constraint Reference (AIR)

To mathematically verify the execution of a WebAssembly payload on an untrusted edge node in Beijing or Moscow, Marabunta utilizes the risc0 zkVM to generate a cryptographic polynomial commitment—a Zero-Knowledge STARK (Scalable Transparent ARguments of Knowledge).

The Hardened WASM Sandbox maps the execution of the wasm32-wasi payload to an Algebraic Intermediate Representation (AIR). This process, known as arithmetization, converts the WASM instructions into a set of polynomial constraints over a finite field.

Implementation: `src/Hardened/zkp.rs`

```
use serde::{Deserialize, Serialize}; use risc0_zkvm::{default_prover, ExecutorEnv, Receipt}; use sha2::{Digest, Sha256};
```

```
#[derive(Debug, Clone, Serialize, Deserialize)]
```

```
pub struct ExecutionProof { /// The serialized risc0 Cryptographic Hash Receipt. pub proof_bytes:
Vec, /// The Image ID (hash) of the RISC-V guest program that verified the trace. pub image_id:
[u32; 8], /// The public inputs (e.g., the hash of the original WASM payload). pub public_inputs:
Vec<[u8; 32]>, }
```

```
pub struct ZkpEngine { // Configuration for the FRI polynomial commitment protocol }

impl ZkpEngine { /// Evaluates the incoming STARK receipt against the known Image ID. /// This
process takes ~12 milliseconds on a standard CPU. pub fn verify_proof(&self, proof:
&ExecutionProof) -> bool { let receipt: Receipt =
bincode::deserialize(&proof.proof_bytes).unwrap();
```

```
    // The mathematical guillotine.
    // If a single constraint in the AIR fails, this returns an
```

```
Err. match receipt.verify(proof.image_id) { Ok(_) => true, // Execution is cryptographically
guaranteed Err(e) => { tracing::error!("Hardened: Cryptographic Hash Verification Failed: {:?}",
e); false } } }
```

13.2 Execution Trace Validation

When a Marabunta node completes a computational task (e.g., a Cooperative tensor gradient update), it generates a dense 1MB STARK receipt. This receipt proves that the node executed the agreed-upon algorithm correctly, without exposing the underlying classified data used during the computation.

The Polynomial Transition Matrix

To generate a valid receipt, the executing node must satisfy hundreds of polynomial constraints at every clock cycle of the RISC-V zkVM.

1. WASM Boot Constraints: The trace must prove that the exact hash of the requested binary (wasm32-wasi) was loaded into the VM's linear memory without modification.
2. Instruction Valid Transitions: The trace must mathematically constrain the Program Counter (PCnext).

Cycle PCcurrent Instruction Mathematical Constraint

100 0x104 ADD PCnext = PCcurrent + 4

101 0x108 JMP 0x200 PCnext = 0x200

102 0x200 TRAP Halt

If a malicious node operator modifies the hypervisor memory mid-execution to inject a biased tensor gradient, the PC transitions will deviate from the expected polynomial curve.

The Harpy Eagle daemon receives the Cryptographic Hash receipt and evaluates the FRI (Fast Reed-Solomon Interactive Oracle Proofs of Proximity) commitment in approximately 12 milliseconds. Because the modified execution trace cannot mathematically satisfy the original AIR constraints, the `receipt.verify()` function fails instantly.

The P2P Split-Brain Paradox (Merkle-DAG Reconciliation)

If a transatlantic fiber cable is physically severed, the Kademlia DHT suffers a catastrophic split-brain partition. The US-Swarm and EU-Swarm will continue mutating their local CRDT states independently.

When the physical connection heals, Marabunta executes Merkle-DAG State Reconciliation. The two divergent halves do not flood the network with millions of CRDT updates. Instead, the Tier-1 Aggregators exchange the cryptographic Merkle Roots of their local state histories. They geometrically traverse the tree downward, isolating the exact branches where divergence occurred, and executing mathematical CRDT merges only on the conflicting deltas, allowing two 50-million node swarms to cleanly suture together in seconds.

The Cryptographic Hash Halting Problem (Deterministic Fuel Metering)

A theoretical paradox arises at the Prover node. According to the Halting Problem, it is statically undecidable whether a given WebAssembly payload contains an infinite loop. If a malicious developer submits a `while(true)` payload, the executing node will run infinitely,

```
attempting to generate an infinitely large STARK polynomial trace
until the host crashes via an
```

Out-Of-Memory (OOM) panic.

Marabunta mathematically bypasses the Halting Problem by embedding Deterministic Fuel Metering directly into the `wasm32-wasi` JIT compiler.

The execution trace is not only constrained by valid Program Counter (`$PC_{next}`) transitions, but by a strictly decreasing thermodynamic "gas limit" defined in the MCL manifest. Every RISC-V instruction consumes a predetermined amount of fuel. If the fuel counter reaches zero before the payload exits, the Wasmtime hypervisor traps the execution deterministically.

The node successfully generates a mathematically valid Cryptographic Hash receipt proving an aborted state transition due to fuel exhaustion. The Harpy Eagle daemon verifies the 12ms proof, the malicious developer's MMX escrow is slashed for the wasted compute, and the node survives without entering an infinite loop.

13.3 Isomorphic State Ring Convergence

To synchronize state globally across heterogeneous hardware (e.g., merging Cooperative gradients from H100s in New York and V100s in Paris) without centralized locking mechanisms, Marabunta uses the Isomorphic State Ring.

The EpidemicStateMap tracks modified tensors and implements true Delta-Only Checkpointing via the `IsomorphicStateRing::flush_to_disk()` function. It asynchronously appends modified Conflict-Free Replicated Data Types (CRDTs) to a Write-Ahead Log (`.wal`), bypassing full-state serialization overhead for sub-millisecond synchronization.

Proof of BFT Finality

A Byzantine node b attempts to inject a conflicting, poisoned state σ' .

In the Marabunta `IsomorphicStateRing`, conflict resolution is handled by the CRDT partial ordering (Last-Writer-Wins semantics on the Nesterov Momentum tensors).

If $\text{hash}(\sigma) > \text{hash}(\sigma')$, the conflict is resolved by the commutativity axiom. The STARK polynomial constraints force the Byzantine node to provide a ZK proof of the state transition that resulted in σ' . Since no valid proof can exist for the malicious state σ' , the node b is mathematically isolated.

Marabunta achieves eventual consistency and Byzantine Fault Tolerance natively, without requiring a centralized sequencer or a slow, synchronous blockchain ledger.

VOLUME 14: THE SOVEREIGNTY OF

CONTROL (BINARY SEGMENTATION)

14.0 The Illusion of Decentralized Chaos

A peer-to-peer swarm scaling to 100 million or more (e.g., 15 billion) nodes presents a profound optical risk to enterprise architects: the illusion of unmanageable chaos. If an infrastructure possesses no centralized AWS control plane to physically unplug, how does a Fortune 500 bank guarantee that a misconfigured algorithm won't permanently saturate the global network or leak proprietary state?

Marabunta resolves this paradox through Asymmetric Binary Segmentation and cryptographic authority.

The Swarm is biologically decentralized at the execution and routing layers, but it is strictly, mathematically authoritarian at the command layer.

14.1 The Monopoly of the Gateway

The Marabunta architecture distributes two fundamentally different binaries.

The software available to the 100 million or more (e.g., 15 billion) public "Ephemeral" nodes—the marabunta-visor—is a neutered execution terminal. It contains the Chrysalis PoW grinder, the Kademia DHT routing logic, and the wasm32-wasi sandbox. Crucially, it physically lacks the Assimilator and Neuromancer orchestration pipelines.

A standard node cannot compile Abstract Syntax Trees (ASTs), it cannot parse Job Control Language (MCL) manifests, and it cannot inject new workloads into the Swarm.

The acquiring enterprise possesses the proprietary Gateway Assimilator binary. This is the only software on Earth containing the ed25519 Root Keys capable of cryptographically signing and injecting a workload that the Harpy daemons will accept. The enterprise maintains an absolute, cryptographic monopoly over the computational capacity of the public mesh.

14.2 Frictionless Infiltration (The Browser Node)

Deploying a custom 14MB Rust binary requiring raw network sockets across a locked-down, 50,000-seat corporate IT fleet is an insurmountable friction point. Enterprise workstations and zero-

trust government laptops will actively block the installation.

Marabunta bypasses OS-level deployment friction entirely.

The marabunta-visor daemon is not just compiled for x86_64-linux or aarch64-apple-darwin. The entire Kademlia routing engine and WASM execution hypervisor compiles down to wasm32-unknown-unknown.

Implementation: src/web_ui/static/worker-exec.js

An enterprise employee simply opens an internal corporate portal in Google Chrome or Microsoft Edge. A background Web Worker instantly boots a full Marabunta node entirely within the browser's V8 JavaScript engine.

```
// The Marabunta Web Worker: Bootstrapping a node without OS installation import init, { WasmNode } from './marabunta_wasm.js';
```

```
async function bootstrapBrowserNode() { await init();
```

```
    // The browser node generates an ephemeral identity and connects
    // to the enterprise Swarm via Secure WebSockets (WSS) or WebRTC.
    const node = new WasmNode();
    await node.connect_to_gateway("wss://gateway.internal.corp");

    // The V8 engine now functions as a Tier-0 Worker, silently
```

```
executing // distributed WASM MapReduce chunks in the background tab.
```

```
node.start_execution_loop(); }
```

The corporation instantly possesses a 50,000-node private supercomputer aggregating data across their global offices. It requires zero administrative privileges, zero .msi or .pkg installers, and zero IT support tickets. The network simply exists wherever a browser tab is open.

14.3 Cryptographic Kill-Switches (Epidemic

Revocation)

If an administrator accidentally deploys a critical financial payload containing an infinite loop or a catastrophic logic error, they cannot SSH into 100 million or more (e.g., 15 billion) edge nodes to kill the processes.

Instead, they execute an Epidemic Revocation.

1. The Poison Pill: The Gateway Administrator cryptographically signs a 256-byte Revocation Certificate with their Root Key, targeting the specific JobId .
2. The Gossip: The certificate is injected into the Kademlia DHT. Utilizing the Plumtree protocol, the kill-order propagates to 100 million or more (e.g., 15 billion) nodes in $O(\log N)$ time (typically under 5 seconds for 15 billion devices).
3. The Guillotine: Every marabunta-visor receives the gossip, cryptographically verifies the Root Key signature against the Genesis Block, and instantly drops the target workload. The daemon flushes the linear memory of the WASM sandbox and permanently erases the .mrbdump journals from the local BlobStore.

The Swarm is autonomous, but the Administrator's cryptographic signature holds absolute, instantaneous veto power over the physics of the network.

ANNEX A: The Sovereign Science Fabric & The Economic Syndicate

A.1 The Geopolitical Context: The Abyssal Treaty

In the era of localized sovereign computing, the monolithic public cloud is an economic bottleneck for fundamental science. The egress fees alone render petabyte-scale physics simulations financially impossible for independent academic and research institutions.

To break this cartel, three nations with deeply aligned petrochemical interests—Brazil, Norway, and the United Kingdom—established the **Abyssal Treaty**. Their shared objective: processing thousands of terabytes of raw acoustic .seggy data to perform Full Waveform Inversion (FWI) on deep-sea seismic reflection models (e.g., the Brazilian Pre-Salt, the Norwegian Barents Sea, and the UK North Sea).

Rather than renting supercomputers, they pooled their sovereign citizen networks into a single, time-zone-arbitraged Marabunta Swarm.

The Three Pillars of the Fabric:

1. **Brazil (The Gamer Grid):** Bundled as an opt-in background daemon within a massively popular indie video game developed by IME/USP alumni. Millions of Brazilian teenagers with high-end, L3-heavy Ryzen CPUs and DDR5 RAM donate compute cycles while they sleep in exchange for MMX micro-credits, which unlock exclusive cosmetic skins.
2. **Norway (The Civic Tax Swarm):** A progressive Scandinavian digital initiative where citizens run the Marabunta daemon on idle MacBooks and home servers. At the end of the fiscal year, cryptographically verified MMX execution receipts are burned in exchange for micro-deductions on their income tax.
3. **The UK (The Night-Shift Academic Net):** Imperial College and allied universities link hundreds of thousands of library desktops and Raspberry Pi lab clusters that sit idle from 8:00 PM to 8:00 AM.

Because the sun never sets on this fabric, thermodynamic efficiency is absolute. When it is 3:00 AM in São Paulo, the cold, sleeping Brazilian grid crunches Norwegian seismic data. When it is 3:00 AM in Oslo, the frozen Scandinavian desktops process Brazilian Pre-Salt models.

However, this environment is inherently hostile. You cannot trust a sleeping teenager's overclocked PC, you cannot trust a library desktop, and you must account for opportunistic script kiddies attempting to defraud the Norwegian tax authority. The Marabunta core architecture was explicitly designed to survive this chaos.

A.2 Phase 1: The Zero-Trust Genesis

To initiate the joint compute session, the lead researchers at USP, NTNU, and Imperial College must establish a cryptographic perimeter.

The USP orchestrator opens the terminal: `mrbswarm init-wolffpack --purpose "Abyssal_FWI_Treaty" --threshold 2-of-3`

Behind the scenes, this is not a mere database entry. The Marabunta CLI triggers the physical **ChrysalisGrinder**. It allocates a 32MB scratchpad and saturates the local L3 cache, executing a memory-hard Proof-of-Work loop until it derives a `NodeId` with 26 leading zero-bits. This mathematically enforces a ~4-second physical cost per identity, neutralizing Sybil botnet attacks.

The CLI then builds the JSON payload and executes an HTTP POST to the local Daemon's `/api/v1/federation/wolffpack` endpoint. The Daemon broadcasts the `SwarmMessage::WolffPackProposal` across the Kademlia DHT. A secure, ephemeral BFT (Byzantine Fault Tolerant) computing coalition is born across three continents.

A.3 Phase 2: Data Gravity & The 1.5TB WASM Bypass

The raw seismic survey block is 1.5 Terabytes. It sits on a secure academic Storage Area Network (SAN) in São Paulo.

A Norwegian citizen's MacBook Air claims a chunk of the wave-equation simulation. Standard WebAssembly (wasm32) imposes a hard 4GB linear memory limit. Naively passing the dataset via the DHT (`SwarmMessage::PushBlob`) would instantly trigger a host-level Out-Of-Memory (OOM) panic, crashing the Swarm.

Instead, the USP JCL manifest specifies a `dataset_shard_uri`.

The Marabunta `WorkEngine` intercepts this URI. Inside the `HighestSecSandbox`, the FWI algorithm calls a custom host-function: `mrbsdataset_stream_read`. The Rust host securely maps the remote SAN file descriptor and streams the 1.5TB dataset across the Atlantic in paginated 1MB windows. The dense wave-equation math executes flawlessly inside the Norwegian

MacBook's WASM sandbox using less than 50MB of RAM, bypassing the architectural limitations of WebAssembly via secure, air-gapped host-calls.

A.4 Phase 3: The "PARANOID" Micro-Audit

FWI math is brutally dense. If a UK library desktop with a spinning HDD attempts to run the simulation, swapping memory pages to disk, it will stall the global pipeline.

To filter the weak hardware, the researchers deploy a custom `verify_mode` labeled `PARANOID`. Before the Orchestrator assigns a heavy wave-equation chunk, the `WorkEngine` forces a pre-flight micro-audit. It demands the bidding node allocate a 256MB matrix, execute a rapid deterministic transformation, and return the cryptographic hash within 2500ms.

A cloud-hosted VM with over-provisioned virtual RAM attempts to swap to a cheap SSD. It fails the 2500ms timer. The Orchestrator instantly rejects the node's bid. Only physical, bare-metal hardware with high-speed DDR5 RAM wins the heavy FWI chunks.

A.5 Phase 4: The Thermal Guillotine

A teenager in London leaves their smartphone charging on the bedside table with the academic daemon running. The acoustic wave math is so intense it pushes the mobile ARM processor to 91°C, risking a catastrophic battery fire.

The `Marabunta HardwareMonitor` daemon continuously polls the device sensors. It detects the critical spike and transitions to `ThermalState::Panic`.

The `WorkEngine` iterates over the active sandboxes and fires `sandbox.kill_engine()`. This forcefully increments the `Wasmtime` epoch, triggering an uncatchable `wasmtimeTrap::Interrupt` from deep outside the synchronous C-level FFI boundary.

The WASM thread is violently physically assassinated. The silicon cools down, preventing physical damage to the citizen's device. The unfinished chunk is gracefully yielded back to the DHT (`SwarmMessage::YieldChunk`) to be computed by a colder node in Brazil.

A.6 Phase 5: The Script Kiddie & The MMX Settlement

A script kiddie in Oslo realizes that the Norwegian government is issuing tax breaks based on Marabunta execution receipts. They write a Python script to spoof the API, instantly returning fake,

randomized FWI stability indices without spending a single CPU cycle on the math.

The USP Orchestrator utilizes `MajorityConsensus`. It routes the exact same FWI chunk to the Norwegian script kiddie and an honest Brazilian gaming PC.

The script kiddie returns `0xBAD . . .` The gamer returns `0x4F2 . . .` The hashes mismatch.

The `VerificationEngine` flags the Norwegian node as an outlier. It executes the `slash_node()` protocol:

1. The node's Kademlia trust score is dropped to `0.0`.
2. It is permanently branded with `Badge::Malicious` in the global `ReputationStore`.
3. Its `NodeId` is blacklisted, isolating it from the DHT.

Finally, the `FederationManager` settles the economy. It executes `settle_verified_work()` based on the `agreed_price` inside the `Assignment` struct. The honest Brazilian gamer is credited with MMX tokens in the SQLite ledger, unlocking their in-game cosmetics. The Norwegian script kiddie earns 0 credits, receiving no tax break, and is permanently exiled from the Sovereign Science Fabric.

VOLUME 16: PLANETARY

ASYMMETRIC DATA ACQUISITION

16.0 The Sovereign Scraping Imperative

Modern scientific and economic models are governed by the volume and quality of their data. To perform massive-scale simulations or market analysis, institutions require continuous, unbounded access to public and specialized data sources across the global internet.

However, the internet is not open. It is actively defended.

If a sovereign research institute attempts to scrape 50 terabytes of data from a protected domain using a centralized IP block (e.g., an AWS datacenter in us-east-1 or an Azure cluster in Europe), they will be immediately neutralized.

Modern Web Application Firewalls (WAFs) and centralized reverse-proxies operate on autonomous, behavioral rate-limiting. They detect the high-velocity requests originating from a unified IP block, categorize the traffic as a "botnet," and issue IP-level bans or endless CAPTCHA challenges, severing the data supply.

Marabunta bypasses these defenses natively. It utilizes the Swarm to execute Planetary Asymmetric Data Acquisition.

16.1 The Ephemeral Edge Advantage

The fundamental architecture of Marabunta is a 100-million node peer-to-peer mesh. Crucially, the vast majority of these nodes (the "Ephemeral" cohort) do not reside in recognizable datacenters. They are consumer laptops, IoT devices, and smartphones operating on residential IP addresses (e.g., Comcast, Vodafone, China Telecom) across 190 countries.

This topology is indistinguishable from organic, human web traffic.

When an Administrator submits an MCL (Marabunta Command Language) payload requesting a massive data extraction operation, the Gateway Assimilator shreds the target URLs into millions of micro-tasks.

```
The Implementation: src/swarm/detection/webserver.rs
```

The Edge nodes execute the HTTP requests directly from their local network interfaces. Because the requests originate from residential ISP IP addresses, they mathematically bypass datacenter IP blocklists.

```
// Execution occurs locally on the Edge Node via the WebServerDetector use
```

```
std::collections::HashMap; use async_trait::async_trait;
```

```
pub struct WebServerDetector { pub host: String, pub port: u16, }
```

```
impl WebServerDetector { /// Executes a distributed web request from a residential edge node. pub
  async fn execute_extraction(&self, url: &str) -> Result<Vec, &'static str> { let response =
    crate::swarm::detection::http_get_raw(&self.host, self.port).await .map_err(|_| "Target unreachable
    from local residential node")?;
```

```
    // The target domain serves the data, unaware that the
```

```
    residential // request is part of a coordinated, global swarm. Ok(response) } }
```

The Ethics of Extration: Consent and Autonomous Throttling

Routing heavy, asymmetric data extraction payloads through the residential IP addresses of unwitting consumers (the "Ephemeral" nodes) introduces severe legal and ethical liabilities. If a target domain detects the scraping pattern and blacklists the IP address, a consumer in Praia Grande or Mumbai could find their home internet banned from accessing standard web services.

```
Marabunta prevents this through Autonomous Legal Throttling (
src/swarm/detection/
```

```
webserver.rs ).
```

1. Opt-In Capability Matrix: A consumer node must explicitly declare its willingness to participate in external web-scraping within its local Marabunta configuration file (config_consent.rs). It is not a default capability.
2. Fractional Saturation: The Swarm routing layer mathematically guarantees that no single residential IP address executes more than 3 HTTP requests to the same target domain within a 24-hour window.
3. The Law of Large Numbers: Because the Swarm possesses 100 million

or more (e.g., 15 billion) potential egress IPs, it can scrape a 1-million page target domain without any single node firing more than a single HTTP GET request.

This renders the scraping traffic functionally indistinguishable from organic human browsing. The target domain's WAF (Web Application Firewall) perceives 1 million unique, geographically distributed human visitors reading one page each. The residential IP is never flagged as a botnet, preserving the integrity of the consumer's connection.

16.2 Bypassing Geographic Subpoenas (Geo-Spoofing)

Certain nation-states or corporate entities employ strict geoblocking. If a target domain is hosted in China and blocks all IP addresses originating from the United States, an American research institution cannot acquire the data using domestic infrastructure.

Marabunta solves this mathematically.

The Administrator writes an MCL manifest with a strict Jurisdiction Fence: `allowed_zones: ["CN", "HK", "MO"]`.

The Swarm routes the extraction WASM payload exclusively to nodes that mathematically prove their physical location within the target jurisdiction via RTT Triangulation (as detailed in Volume 05).

The Chinese target domain receives HTTP requests originating from organic, residential Chinese IP addresses (e.g., a laptop in Shenzhen). The domain serves the data. The Shenzhen node processes the HTML, extracts the required payload, encrypts it via the Sphinx protocol, and routes it back through the Phantom Overlay to the United States.

16.3 Kinetic DDoS Deflection and the "Fluid CDN"

The inverse is also true. Marabunta provides ultimate resilience for content hosted within the Swarm, effectively serving as an Un-Censorable Content Delivery Network (CDN) that bypasses centralized vendors.

If a sovereign government issues a subpoena to a centralized DNS provider to seize a domain, or if a botnet launches a 500Gbps DDoS attack against a specific server, traditional architectures collapse.

In Marabunta, static assets are shredded into Reed-Solomon chunks and injected into the Kademlia DHT. Users access the content via its cryptographic hash (`blake3`) over the Sphinx UDP overlay.

Stigmergic Evaporation & Load Dissipation

How does a network without a center survive a DDoS attack?

If a botnet floods the Swarm with requests for a specific cryptographic hash, the Wild Dogs daemon detects the artificial latency spike in that specific XOR bucket. The Kademlia routing table dynamically shatters.

The Crocodile and Plumtree daemons clone the requested data chunks to 100,000 surrounding nodes in milliseconds to absorb the impact. The harder the botnet hits the target hash, the more the Swarm replicates the target to dissipate the thermal load.

The attack energy is literally weaponized to strengthen the network's availability.